University of the Philippines - Baguio

Team - updog

30/04/2026

Contents

1. Data Structures 1
1.1. Fenwick Tree 1
1.2. Segment Tree 1
1.3. Union Find 1
1.4. Leq Counter 1
1.4.1. Leq Counter Array 1
1.4.2. Leq Counter Map 1
1.5. Misof Tree 1
1.6. Mo Algorithm 2
1.7. Ordered Statistics Tree 2
1.8. Splay Tree 2
1.9. Implicit Treap 3
1.10. Unique Counter 4
2. Dynamic Programming 4
2.1. Lichao Tree 4
2.2. Divide And Conquer 4
3. Geometry 5
3.1. Angles And Rotations 5
3.2. Distances 5
3.3. Intersections 5
3.3.1. Circle Line Intersection 5
3.3.2. Circle Circle Intersection 5
3.3.3. Line Intersect 5
3.4. Polygon Centroid 5
3.5. Convex Hull 5
3.5.1. Convex Hull 5
3.5.2. Convex Hull 3D 5
3.6. Areas 5
3.6.1. Triangle Area 5
3.6.2. Polygon Area 6
3.7. Dot Cross 6
3.8. Line Sweep Closest Pair 6
3.9. Point Projection 6
3.10. Great Circle Distance 6
3.11. Point In Polygon 6
3.12. Cut Polygon By Line 7
3.13. Triangle Centers 7
3.14. Convex Polygon Intersection 7
3.15. Pick Theorem Lattice 8
3.16. Minimum Enclosing Circle 8
3.17. Shamos Algorithm 8
3.18. Kd Tree 8
4. Graphs 8
4.1. Max Flow 8
4.1.1. Dinic 8
4.2. Shortest Path 9
4.2.1. Dijkstra 9
4.2.2. Bellman Ford 9
4.3. Minimum Spanning Tree 9
4.3.1. Kruskal Min Spanning Tree 9
4.3.2. Prim Min Spanning Tree 9
4.4. Least Common Ancestor 9
4.4.1. Binary Lifting Lca 9
4.4.2. Euler Lca 10
4.4.3. Tarjan Lca 10
4.5. Minimum Mean Weight Cycle 10
4.6. Strongly Connected Components 10
4.7. Heavy Light Decomposition 10
4.8. Euler Path/Cycle 11
4.8.1. Directed Euler 11
4.8.2. Undirected Euler 11
4.9. Tree Isomorphism 12
5. Math I Algebra 12
5.0.1. Generating Function Manager 12
5.0.2. Fast Fourier Transform 12
5.0.3. Fft Polynomial Multiplication 13
5.0.4. Polynomial Long Division 13
5.0.5. Matrix Multiplication 13
5.0.6. Matrix Power 13
5.0.7. Gauss Jordan Matrix Determinant 14
6. Math II Combinatorics 14
6.0.1. Lucas Theorem 14
6.0.2. Catalan Numbers 14
6.0.3. Kth Permutation 14
6.0.4. Factoradics 14
7. Math III Number Theory 14
7.0.1. Gcd Subset Counter 14
7.0.2. Linear Diophantine 14
7.0.3. Chinese Remainder Theorem 15
7.0.4. Euler Totient 15
7.0.5. Modular Exponentiation 15
7.0.6. Mod Inverse 15
7.0.7. Modulo Solver 15
7.0.8. Primitive Root 15
8. Math IV Numerical Methods 15
8.0.1. Fast Square Testing 15
8.0.2. Simpson Integration 15
9. Strings 15
9.1. Longest Common Prefix 15
9.2. Hasher 16
9.3. Z Algorithm 16
9.4. Aho Corasick Trie 16
9.5. Palindromes 16
9.5.1. Palindromic Tree 16
9.5.2. Eertree 17
9.6. Trie 17
9.7. Suffix Array 17
10. Other Algorithms 18
10.1. Longest Increasing Subsequence 18
10.2. Cycle Find 18
10.3. Stable Marriage 18
10.4. Dates 18
10.5. Simulated Annealing 18
10.6. Simplex 18
10.7. Fast Input Reading 19
10.8. Bit Hacks 19
10.9. Physics 19
10.10. Markov Chains 19
10.11. Primitive Roots 19
10.12. Floor 19
10.13. Maximum-Weight Closure 19
10.14. Tutte Matrix 19
10.15. Spanning Trees 20
10.16. Möbius inversion 20
10.17. Integer points 20

1. Data Structures

1.1. Fenwick Tree.

```
class Fenwick:
--- def __init__(self, arr):
------- self.n = len(arr)
------- self.ar = [0] * self.n
------- for i in range(self.n):
----------- self.ar[i] += arr[i]
----------- j = i | (i + 1)
----------- if j < self.n:
--------------- self.ar[j] += self.ar[i]

--- def sum(self, i):
------- res = 0
------- while i >= 0:
----------- res += self.ar[i]
----------- i = (i & (i + 1)) - 1
------- return res

--- def range_sum(self, i, j):
------- return self.sum(j) - (self.sum(i - 1) if i > 0 else 0)

--- def add(self, i, val):
```

```
        while i < self.n:
            self.ar[i] += val
            i |= i + 1

    def get(self, i):
        res = self.ar[i]
        if i > 0:
            lca = (i & (i + 1)) - 1
            i -= 1
            while i != lca:
                res -= self.ar[i]
                i = (i & (i + 1)) - 1
        return res

    def set(self, i, val):
        self.add(i, val - self.get(i))

    def range_add(self, l, r, val):
        self.add(l, val)
        if r + 1 < self.n:
            self.add(r + 1, -val)

    def point_query(self, i):
        return self.sum(i)
```

1.2. Segment Tree.

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (2 * self.n)
        for i in range(self.n):
            self.tree[self.n + i] = data[i]
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[i << 1] + self.tree[i << 1 | 1]

    def update(self, p, value):
        p += self.n
        self.tree[p] = value
        while p > 1:
            self.tree[p >> 1] = self.tree[p] + self.tree[p ^ 1]
            p >>= 1

    def query(self, l, r):
        res = 0
        l += self.n
        r += self.n
        while l < r:
            if l & 1:
                res += self.tree[l]
                l += 1
            if r & 1:
                r -= 1
                res += self.tree[r]
            l >>= 1
            r >>= 1
        return res
```

1.3. Union Find.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)

        if root_i != root_j:
            if self.size[root_i] < self.size[root_j]:
                root_i, root_j = root_j, root_i
            self.parent[root_j] = root_i
            self.size[root_i] += self.size[root_j]
            return True
        return False

    def get_size(self, i):
        return self.size[self.find(i)]
```

1.4. Leq Counter.

1.4.1. *Leq Counter Array.*

```
def merge(left, right):
    i = j = 0
    res = []
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            res.append(left[i])
            i += 1
        else:
            res.append(right[j])
            j += 1
    res.extend(left[i:])
    res.extend(right[j:])
    return res
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

1.4.2. *Leq Counter Map.*

```
import bisect

class MergeSortTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.size = 1
        while self.size < self.n:
            self.size <<= 1

        self.tree = [[] for _ in range(2 * self.size)]

        for i in range(self.n):
            self.tree[self.size + i] = [arr[i]]

        for i in range(self.size - 1, 0, -1):
            self.tree[i] = sorted(self.tree[i << 1] + self.tree[i << 1
↪ | 1])

    def query(self, l, r, x):
        l += self.size
        r += self.size

        ans = 0

        while l < r:
            if l & 1:
                ans += bisect.bisect_right(self.tree[l], x)
                l += 1
            if r & 1:
                r -= 1
                ans += bisect.bisect_right(self.tree[r], x)
            l >>= 1
            r >>= 1

        return ans
```

1.5. Misof Tree.

```
BITS = 15
MAXV = 1 << BITS

class MisofTree:
    def __init__(self):
        self.cnt = [[0, 0] for _ in range(1 << (BITS + 1))]
        self.root = 1
        self.nodes = 2

    def _new_node(self):
        idx = self.nodes
        self.nodes += 1
        if idx >= len(self.cnt):
            self.cnt.append([0, 0])
        return idx

    def insert(self, x):
        node = self.root
        for i in reversed(range(BITS)):
            b = (x >> i) & 1
            self.cnt[node][b] += 1

            nxt = self.cnt[node].__len__()
            if not hasattr(self, "child"):
                self.child = {}

            if (node, b) not in self.child:
                self.child[(node, b)] = self._new_node()

            node = self.child[(node, b)]

    def erase(self, x):
        node = self.root
        path = []

        for i in reversed(range(BITS)):
            b = (x >> i) & 1
            path.append((node, b))
            node = self.child[(node, b)]

        for node, b in path:
            self.cnt[node][b] -= 1

    def kth(self, k):
        node = self.root
        res = 0

        for i in reversed(range(BITS)):
            left = self.cnt[node][0]

            if k < left:
                node = self.child.get((node, 0), node)
            else:
                k -= left
                res |= (1 << i)
                node = self.child.get((node, 1), node)

        return res
```

1.6. Mo Algorithm.

```
from collections import defaultdict

LOGN = 21

def hilbert_order(x, y, pow, rotate):
    if pow == 0:
        return 0
    hpow = 1 << (pow - 1)
    if x < hpow:
        if y < hpow:
            seg = 0
        else:
            seg = 3
    else:
        if y < hpow:
            seg = 1
        else:
            seg = 2
    seg = (seg + rotate) & 3
    rotate_delta = [3, 0, 0, 1]
    nx = x & (x ^ hpow)
    ny = y & (y ^ hpow)
    nrot = (rotate + rotate_delta[seg]) & 3
    sub_sq_size = 1 << (2 * pow - 2)
    ans = seg * sub_sq_size
    add = hilbert_order(nx, ny, pow - 1, nrot)
    if seg == 1 or seg == 2:
        ans += add
    else:
        ans += sub_sq_size - add - 1
    return ans

class Query:
    def __init__(self, idx, l, r):
        self.id = idx
        self.l = l
        self.r = r
        self.ord = hilbert_order(l, r, LOGN, 0)

    def __lt__(self, other):
        return self.ord < other.ord

def mo_algorithm(arr, queries):
    queries.sort()

    freq = defaultdict(int)
    distinct = 0

    def add(i):
        nonlocal distinct
        x = arr[i]
        freq[x] += 1
        if freq[x] == 1:
            distinct += 1

    def remove(i):
        nonlocal distinct
        x = arr[i]
        freq[x] -= 1
        if freq[x] == 0:
            distinct -= 1
    ans = [0] * len(queries)
    l = 0
    r = -1
    for q in queries:
        while r < q.r:
            r += 1
            add(r)
        while r > q.r:
            remove(r)
            r -= 1
        while l > q.l:
            l -= 1
            add(l)
        while l < q.l:
            remove(l)
            l += 1
        ans[q.id] = distinct
    return ans
```

1.7. Ordered Statistics Tree.

```
class OrderedSet:
    def __init__(self):
        self.root = None

    def insert(self, x):
        if self.contains(x):
            return
        l, r = split(self.root, x)
        self.root = merge(merge(l, Node(x)), r)

    def erase(self, x):
        self.root = self._erase(self.root, x)

    def _erase(self, t, x):
        if not t:
            return None
        if t.key == x:
            return merge(t.left, t.right)
        elif x < t.key:
            t.left = self._erase(t.left, x)
        else:
            t.right = self._erase(t.right, x)
        pull(t)
        return t

    def contains(self, x):
        t = self.root
        while t:
            if t.key == x:
                return True
            t = t.left if x < t.key else t.right
        return False

    def order_of_key(self, x):
        t = self.root
        res = 0
        while t:
            if x <= t.key:
                t = t.left
            else:
                res += 1 + size(t.left)
                t = t.right
        return res

    def find_by_order(self, k):
        t = self.root
        while t:
            left_sz = size(t.left)
            if k < left_sz:
                t = t.left
            elif k == left_sz:
                return t.key
            else:
                k -= left_sz + 1
                t = t.right
        return None
```

1.8. Splay Tree.

```
class SplayTree:
    def __init__(self, arr=None):
        self.root = None
        if arr:
            self.root = self.build(arr, 0, len(arr))
    def build(self, arr, l, r):
        if l >= r:
            return None
        m = (l + r) // 2
        node = Node(arr[m])
        node.left = self.build(arr, l, m)
        node.right = self.build(arr, m + 1, r)
        if node.left:
            node.left.parent = node
        if node.right:
            node.right.parent = node
        self.pull(node)
        return node
    def pull(self, x):
        x.size = 1
        if x.left:
            x.size += x.left.size
        if x.right:
            x.size += x.right.size
    def push(self, x):
        if x and x.rev:
            x.left, x.right = x.right, x.left
            if x.left:
                x.left.rev ^= True
            if x.right:
                x.right.rev ^= True
            x.rev = False
    def rotate(self, x):
        p = x.parent
        g = p.parent

        if p.left == x:
            b = x.right
            x.right = p
            p.left = b
        else:
            b = x.left
            x.left = p
            p.right = b
        if b:
            b.parent = p
        x.parent = g
        if g:
            if g.left == p:
                g.left = x
            else:
                g.right = x
        p.parent = x
        self.pull(p)
        self.pull(x)
    def splay(self, x):
        if not x:
            return None

        while x.parent:
            p = x.parent
            g = p.parent

            if g:
                self.push(g)
            self.push(p)
            self.push(x)

            if not g:
                self.rotate(x)
            elif (g.left == p) == (p.left == x):
                self.rotate(p)
                self.rotate(x)
            else:
```

```
---------------- self.rotate(x) ----------------------------------------
---------------- self.rotate(x) ----------------------------------------
------------------------------------------------------------------------
------- self.root = x ----------------------------------------------------
------- return x ---------------------------------------------------------
------------------------------------------------------------------------
--- def kth(self, k): ------------------------------------------------
------- x = self.root ----------------------------------------------------
------- while x: ---------------------------------------------------------
----------- self.push(x) -------------------------------------------------
----------- lsize = x.left.size if x.left else 0 -------------------------
------------------------------------------------------------------------
----------- if k < lsize: ------------------------------------------------
---------------- x = x.left ---------------------------------------------
----------- elif k > lsize: ----------------------------------------------
---------------- k -= lsize + 1 -----------------------------------------
---------------- x = x.right --------------------------------------------
----------- else: --------------------------------------------------------
---------------- return self.splay(x) -----------------------------------
------- return None ------------------------------------------------------
------------------------------------------------------------------------
--- def split(self, k): ----------------------------------------------
------- if k == 0: -------------------------------------------------------
----------- r = self.root ------------------------------------------------
----------- self.root = None ---------------------------------------------
----------- return None, r -----------------------------------------------
------------------------------------------------------------------------
------- x = self.kth(k - 1) ----------------------------------------------
------------------------------------------------------------------------
------- right = x.right --------------------------------------------------
------- if right: --------------------------------------------------------
----------- right.parent = None ------------------------------------------
------------------------------------------------------------------------
------- x.right = None ---------------------------------------------------
------- self.pull(x) -----------------------------------------------------
------------------------------------------------------------------------
------- return x, right --------------------------------------------------
------------------------------------------------------------------------
--- def merge(self, left, right): ------------------------------------
------- if not left: -----------------------------------------------------
----------- return right -------------------------------------------------
------- if not right: ----------------------------------------------------
----------- return left --------------------------------------------------
------------------------------------------------------------------------
------- x = left ---------------------------------------------------------
------- while True: ------------------------------------------------------
----------- self.push(x) -------------------------------------------------
----------- if not x.right: ----------------------------------------------
---------------- break --------------------------------------------------
----------- x = x.right --------------------------------------------------
------------------------------------------------------------------------
------- while x.parent: --------------------------------------------------
----------- if x.parent.parent: ------------------------------------------
---------------- if (x.parent.left == x) == (x.parent.parent.left ==
↪  x.parent):
-------------------- self.rotate(x.parent) ---------------------------
---------------- else: --------------------------------------------------
-------------------- self.rotate(x) ----------------------------------
----------- self.rotate(x) -----------------------------------------------
------------------------------------------------------------------------
------- x.right = right --------------------------------------------------
------- right.parent = x -------------------------------------------------
------- self.pull(x) -----------------------------------------------------
------------------------------------------------------------------------
------- return x ---------------------------------------------------------
------------------------------------------------------------------------
--- def insert(self, k, val): ----------------------------------------
------- left, right = self.split(k) --------------------------------------
------- node = Node(val) -------------------------------------------------
------- merged = self.merge(left, node) ----------------------------------
------- self.root = self.merge(merged, right) ----------------------------
------------------------------------------------------------------------
--- def _split_root(self, root, k): ----------------------------------
------- if not root: -----------------------------------------------------
----------- return None, None --------------------------------------------
------- self.root = root -------------------------------------------------
------- return self.split(k) ---------------------------------------------
------------------------------------------------------------------------
--- def erase(self, k): ----------------------------------------------
------- left, temp = self.split(k) ---------------------------------------
------- mid, right = self._split_root(temp, 1) ---------------------------
------- self.root = self.merge(left, right) ------------------------------
------------------------------------------------------------------------
--- def reverse(self, l, r): -----------------------------------------
------- left, temp = self.split(l) ---------------------------------------
------- mid, right = self._split_root(temp, r - l + 1) -------------------
------------------------------------------------------------------------
------- if mid: ----------------------------------------------------------
----------- mid.rev ^= True ----------------------------------------------
------------------------------------------------------------------------
------- merged = self.merge(left, mid) -----------------------------------
------- self.root = self.merge(merged, right) ----------------------------
------------------------------------------------------------------------
--- def assign(self, k, val): ----------------------------------------
------- node = self.kth(k) -----------------------------------------------
------- node.value = val -------------------------------------------------
------- self.pull(node) --------------------------------------------------
------------------------------------------------------------------------
--- def inorder(self): -----------------------------------------------
------- res = [] ---------------------------------------------------------
------- stack = [] -------------------------------------------------------
------- x = self.root ----------------------------------------------------
------------------------------------------------------------------------
------- while stack or x: ------------------------------------------------
----------- while x: -----------------------------------------------------
---------------- self.push(x) -------------------------------------------
---------------- stack.append(x) ----------------------------------------
---------------- x = x.left ---------------------------------------------
------------------------------------------------------------------------
----------- x = stack.pop() ----------------------------------------------
----------- res.append(x.value) ------------------------------------------
----------- x = x.right --------------------------------------------------
------------------------------------------------------------------------
------- return res -------------------------------------------------------
------------------------------------------------------------------------
--- def check_no_cycle(self): ----------------------------------------
------- seen = set() -----------------------------------------------------
------------------------------------------------------------------------
------- def dfs(x): ------------------------------------------------------
----------- if not x: ----------------------------------------------------
---------------- return -------------------------------------------------
----------- if id(x) in seen: --------------------------------------------
---------------- raise Exception("Cycle detected") ----------------------
----------- seen.add(id(x)) ----------------------------------------------
----------- dfs(x.left) --------------------------------------------------
----------- dfs(x.right) -------------------------------------------------
------------------------------------------------------------------------
------- dfs(self.root) ---------------------------------------------------
```

1.9. Implicit Treap.

```
class Node: ------------------------------------------------------------
--- __slots__ = ("val", "sum", "delta", "prio", "size", "l", "r") ----
------------------------------------------------------------------------
--- def __init__(self, val): -----------------------------------------
------- self.val = val ---------------------------------------------------
------- self.sum = val ---------------------------------------------------
------- self.delta = 0 ---------------------------------------------------
------- self.prio = random.randint(1, 1 << 30) ---------------------------
------- self.size = 1 ----------------------------------------------------
------- self.l = None ----------------------------------------------------
------- self.r = None ----------------------------------------------------
------------------------------------------------------------------------
def sz(v): -------------------------------------------------------------
--- return v.size if v else 0 ----------------------------------------
------------------------------------------------------------------------
------------------------------------------------------------------------
def sm(v): -------------------------------------------------------------
--- return v.sum if v else 0 -----------------------------------------
------------------------------------------------------------------------
------------------------------------------------------------------------
def apply(v, d): -------------------------------------------------------
--- if not v: --------------------------------------------------------
------- return -----------------------------------------------------------
--- v.delta += d -----------------------------------------------------
--- v.val += d -------------------------------------------------------
--- v.sum += d * sz(v) -----------------------------------------------
------------------------------------------------------------------------
------------------------------------------------------------------------
def push(v): -----------------------------------------------------------
--- if not v or v.delta == 0: ----------------------------------------
------- return -----------------------------------------------------------
--- apply(v.l, v.delta) ----------------------------------------------
--- apply(v.r, v.delta) ----------------------------------------------
--- v.delta = 0 ------------------------------------------------------
------------------------------------------------------------------------
def pull(v): -----------------------------------------------------------
--- if not v: --------------------------------------------------------
------- return -----------------------------------------------------------
--- v.size = 1 + sz(v.l) + sz(v.r) -----------------------------------
--- v.sum = v.val + sm(v.l) + sm(v.r) --------------------------------
------------------------------------------------------------------------
def split(v, k): -------------------------------------------------------
--- if not v: --------------------------------------------------------
------- return (None, None) ----------------------------------------------
------------------------------------------------------------------------
--- push(v) ----------------------------------------------------------
------------------------------------------------------------------------
--- if sz(v.l) >= k: -------------------------------------------------
------- l, v.l = split(v.l, k) -------------------------------------------
------- pull(v) ----------------------------------------------------------
------- return (l, v) ----------------------------------------------------
--- else: ------------------------------------------------------------
------- v.r, r = split(v.r, k - sz(v.l) - 1) -----------------------------
------- pull(v) ----------------------------------------------------------
------- return (v, r) ----------------------------------------------------
------------------------------------------------------------------------
def merge(l, r): -------------------------------------------------------
--- if not l or not r: -----------------------------------------------
------- return l or r ----------------------------------------------------
------------------------------------------------------------------------
--- if l.prio > r.prio: ----------------------------------------------
------- push(l) ----------------------------------------------------------
------- l.r = merge(l.r, r) ----------------------------------------------
------- pull(l) ----------------------------------------------------------
------- return l ---------------------------------------------------------
--- else: ------------------------------------------------------------
------- push(r) ----------------------------------------------------------
------- r.l = merge(l, r.l) ----------------------------------------------
------- pull(r) ----------------------------------------------------------
------- return r ---------------------------------------------------------
------------------------------------------------------------------------
class Treap: -----------------------------------------------------------
--- def __init__(self): ----------------------------------------------
------- self.root = None -------------------------------------------------
------------------------------------------------------------------------
--- def size(self): --------------------------------------------------
------- return sz(self.root) ---------------------------------------------
------------------------------------------------------------------------
--- def insert(self, pos, val): --------------------------------------
------- node = Node(val) -------------------------------------------------
------- l, r = split(self.root, pos) -------------------------------------
------- self.root = merge(merge(l, node), r) -----------------------------
------------------------------------------------------------------------
--- def erase(self, pos): --------------------------------------------
------- l, r = split(self.root, pos) -------------------------------------
```

```
        m, r = split(r, 1)
        self.root = merge(l, r)

    def get(self, pos):
        def _get(v, k):
            push(v)
            if k < sz(v.l):
                return _get(v.l, k)
            elif k > sz(v.l):
                return _get(v.r, k - sz(v.l) - 1)
            return v.val
        return _get(self.root, pos)

    def range_sum(self, lq, rq):
        l1, r1 = split(self.root, rq + 1)
        l2, mid = split(l1, lq)

        res = sm(mid)

        self.root = merge(merge(l2, mid), r1)
        return res

    def range_add(self, lq, rq, d):
        l1, r1 = split(self.root, rq + 1)
        l2, mid = split(l1, lq)

        apply(mid, d)

        self.root = merge(merge(l2, mid), r1)

    def inorder(self):
        res = []

        def dfs(v):
            if not v:
                return
            push(v)
            dfs(v.l)
            res.append(v.val)
            dfs(v.r)

        dfs(self.root)
        return res
```

1.10. Unique Counter.

```
import bisect

class LeqCounter:
    def __init__(self, arr):
        self.n = len(arr)
        self.seg = [[] for _ in range(4 * self.n)]
        self._build(1, 0, self.n - 1, arr)

    def _build(self, idx, l, r, arr):
        if l == r:
            self.seg[idx] = [arr[l]]
            return
        m = (l + r) // 2
        self._build(idx * 2, l, m, arr)
        self._build(idx * 2 + 1, m + 1, r, arr)

        # merge
        self.seg[idx] = sorted(self.seg[idx * 2] + self.seg[idx * 2 +
↪ 1])

    def _query(self, idx, l, r, ql, qr, x):
        if qr < l or r < ql:
            return 0
        if ql <= l and r <= qr:
            return bisect.bisect_right(self.seg[idx], x)

        m = (l + r) // 2
        return (
            self._query(idx * 2, l, m, ql, qr, x) +
            self._query(idx * 2 + 1, m + 1, r, ql, qr, x)
        )

    def count(self, l, r, x):
        if l > r:
            return 0
        return self._query(1, 0, self.n - 1, l, r, x)

class UniqueCounter:
    def __init__(self, A):
        n = len(A)
        self.B = [0] * (n + 1)
        last = {}

        for i in range(1, n + 1):
            val = A[i - 1]
            self.B[i] = last.get(val, 0)
            last[val] = i
        self.leq = LeqCounter(self.B)

    def count(self, l, r):
        return self.leq.count(l + 1, r + 1, l)
```

2. Dynamic Programming

2.1. Lichao Tree.

```
class Line:
    __slots__ = ['m', 'c']
    def __init__(self, m=0, c=float('-inf')):
        self.m = m
        self.c = c

    def eval(self, x):
        return self.m * x + self.c

class Node:
    __slots__ = ['line', 'left', 'right']
    def __init__(self):
        self.line = Line()
        self.left = None
        self.right = None

class LiChaoTree:
    def __init__(self, min_x=-10**12, max_x=10**12):
        self.root = Node()
        self.min_x = min_x
        self.max_x = max_x

    def add_line(self, m, c):
        self._insert(self.root, self.min_x, self.max_x, Line(m, c))

    def _insert(self, node, l, r, new_line):
        mid = (l + r) // 2

        left_better = new_line.eval(l) > node.line.eval(l)
        mid_better = new_line.eval(mid) > node.line.eval(mid)

        if mid_better:
            node.line, new_line = new_line, node.line
        if r - l <= 1:
            return

        if left_better != mid_better:
            if not node.left:
                node.left = Node()
            self._insert(node.left, l, mid, new_line)
        else:
            if not node.right:
                node.right = Node()
            self._insert(node.right, mid, r, new_line)

    def query(self, x):
        return self._query(self.root, self.min_x, self.max_x, x)

    def _query(self, node, l, r, x):
        if not node:
            return float('-inf')
        if r - l <= 1:
            return node.line.eval(x)

        mid = (l + r) // 2
        res = node.line.eval(x)

        if x < mid:
            res = max(res, self._query(node.left, l, mid, x))
        else:
            res = max(res, self._query(node.right, mid, r, x))

        return res
```

2.2. Divide And Conquer.

```
INF = float('inf')

def solve_dp(g, k_L, k_R, n_L, n_R, dp, cost_func):
    if n_L > n_R:
        return

    n_M = (n_L + n_R) // 2
    best_k = -1
    dp[g][n_M] = INF

    for k in range(k_L, min(n_M, k_R) + 1):
        current_val = dp[g - 1][k] + cost_func(k + 1, n_M)
        if current_val < dp[g][n_M]:
            dp[g][n_M] = current_val
            best_k = k

    solve_dp(g, k_L, best_k, n_L, n_M - 1, dp, cost_func)
    solve_dp(g, best_k, k_R, n_M + 1, n_R, dp, cost_func)
```

3. Geometry

3.1. Angles And Rotations.

```
def angle(a, b, c): # angle formed by abc in radians: PI < x <= PI
    return abs((arg(a-b) - arg(c-b)) % (2 * math.pi))
def rotate(p, a, d): # rotate point a about pivot p CCW at d radians
    return p + (a - p) * complex(math.cos(d), math.sin(d))
```

3.2. Distances.

```
# Distance: Point to 2D Line (ax + by + c = 0)
def dist_pt_line_eq(p, a, b, c):
    return abs(a*p.real + b*p.imag + c) / math.hypot(a, b)

# Distance: Point to 2D Line through points a and b
def dist_pt_line(p, a, b):
    cp = (p - a).conjugate() * (b - a)
    return abs(cp.imag) / abs(b - a)

# Distance: Point to 3D Plane (ax + by + cz + d = 0)
def dist_pt_plane(p, a, b, c, d):
    # math.hypot supports 3+ arguments in Python 3.8+
    return abs(a*p.x + b*p.y + c*p.z + d) / math.hypot(a,b,c)

# Distance: Shortest distance between 3D lines AB and CD
def dist_line_3d(A, B, C, D):
    u, v, w = B - A, D - C, A - C
```

```
--- a, b = u.dot(u), u.dot(v) ----------------------------------------
--- c, d, e = v.dot(v), u.dot(w), v.dot(w) ---------------------------
--- det = a * c - b * b ----------------------------------------------
--- if det < 1e-9: # Parallel lines ----------------------------------
------- s, t = 0.0, d / b if b > c else e / c ----------------------
--- else: # Skew lines -----------------------------------------------
------- s = (b * e - c * d) / det ----------------------------------
------- t = (a * e - b * d) / det ----------------------------------
--- p1 = P3(A.x + u.x*s, A.y + u.y*s, A.z + u.z*s) -------------------
--- p2 = P3(C.x + v.x*t, C.y + v.y*t, C.z + v.z*t) -------------------
--- vec = p1 - p2 ----------------------------------------------------
--- return math.sqrt(vec.dot(vec)) -----------------------------------
```

3.3. Intersections.

3.3.1. *Circle Line Intersection.*

```
EPS = 1e-9 -------------------------------------------------------------
def projLine(p, a, b): # Project point p onto line segment ab --------
--- v = b - a --------------------------------------------------------
--- scalar = ((p - a) * v.conjugate()).real / abs(v)**2 -------------
--- return a + v * scalar --------------------------------------------
 ----------------------------------------------------------------------
def rotate(c, p, t): # Rotate point p around center c by t radians ---
--- return c + (p - c) * cmath.rect(1, t) ----------------------------
 ----------------------------------------------------------------------
def CL_inter(c, r, a, b): ----------------------------------------------
--- p = projLine(c, a, b) --------------------------------------------
--- d = abs(c - p) ---------------------------------------------------
--- ans = [] ---------------------------------------------------------
--- ----------------------------------------------------------------
--- if d > r + EPS: pass # None --------------------------------------
--- elif d > r - EPS: ans.append(p) # Tangent ------------------------
--- elif d < EPS: # Diameter -----------------------------------------
------- v = r * (b - a) / abs(b - a) -------------------------------
------- ans.append(c + v) ------------------------------------------
------- ans.append(c - v) ------------------------------------------
--- else: # Secant ---------------------------------------------------
------- t = math.acos(d / r) ---------------------------------------
------- p = c + (p - c) * r / d ------------------------------------
------- ans.append(rotate(c, p, t)) --------------------------------
------- ans.append(rotate(c, p, -t)) -------------------------------
--- return ans -------------------------------------------------------
```

3.3.2. *Circle Circle Intersection.*

```
EPS = 1e-9 -------------------------------------------------------------
def rotate(c, p, t): ---------------------------------------------------
--- # Rotate point p around center c by t radians --------------------
--- return c + (p - c) * cmath.rect(1, t) ----------------------------
 ----------------------------------------------------------------------
def CC_inter(c1, r1, c2, r2): ------------------------------------------
--- d = abs(c1 - c2) -------------------------------------------------
--- ans = [] ---------------------------------------------------------
--- if d < EPS: # Concentric circles ---------------------------------
------- if abs(r1 - r2) < EPS: -------------------------------------
----------- pass # Infinite intersections (same circle) --------------
--- elif r1 < EPS: # Circle 1 is a point -----------------------------
------- if abs(d - r2) < EPS: --------------------------------------
----------- ans.append(c1) -------------------------------------------
--- else: # General case using Law of Cosines -------------------------
------- s = (r1*r1 + d*d - r2*r2) / (2*r1*d) -----------------------
------- s = max(-1.0, min(1.0, s)) # Prevent float domain errors -----
------- t = math.acos(s) -------------------------------------------
------- # Point on C1's radius pointing towards C2 -------------------
------- mid = c1 + (c2 - c1) * r1 / d ------------------------------
------- ans.append(rotate(c1, mid, t)) -----------------------------
------- if abs(math.sin(t)) >= EPS: --------------------------------
----------- # FIXED: Rotate around c1, not c2! ----------------------
----------- ans.append(rotate(c1, mid, -t)) ------------------------
--- return ans -------------------------------------------------------
```

3.3.3. *Line Intersect.*

```
def line_inter_fast(a, b, c, d): ---------------------------------------
--- A = cross(d - a, b - a) ------------------------------------------
--- B = cross(c - a, b - a) ------------------------------------------
--- return (B * d - A * c) / (B - A) if B - A else None --------------
```

3.4. Polygon Centroid.

```
def cross(a, b): -------------------------------------------------------
--- return a[0] * b[1] - a[1] * b[0] ---------------------------------
 ----------------------------------------------------------------------
def centroid(points): --------------------------------------------------
--- n = len(points) --------------------------------------------------
--- ans_x, ans_y = 0.0, 0.0 ------------------------------------------
--- z = 0.0 ----------------------------------------------------------
 ----------------------------------------------------------------------
--- for i in range(n): -----------------------------------------------
------- j = (i - 1) % n --------------------------------------------
------- cp = cross(points[j], points[i]) ---------------------------
 ----------------------------------------------------------------------
------- ans_x += (points[j][0] + points[i][0]) * cp ----------------
------- ans_y += (points[j][1] + points[i][1]) * cp ----------------
------- z += cp ----------------------------------------------------
 ----------------------------------------------------------------------
--- return (ans_x / (3 * z), ans_y / (3 * z)) ------------------------
```

3.5. Convex Hull.

3.5.1. *Convex Hull.*

```
def cross(o, a, b): # 2D cross product of OA and OB vectors ----------
--- return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0]) ---------
 ----------------------------------------------------------------------
def convex_hull(pts): # Sorts by x, then by y automatically ----------
--- pts = sorted(set(pts))  ------------------------------------------
--- if len(pts) <= 1: ------------------------------------------------
------- return pts -------------------------------------------------
--- def build(iterable): ---------------------------------------------
------- h = [] -----------------------------------------------------
------- for p in iterable: -----------------------------------------
----------- while len(h) >= 2 and cross(h[-2], h[-1], p) <= 0: -------
--------------- h.pop() ------------------------------------------
----------- h.append(p) ----------------------------------------------
------- return h ---------------------------------------------------
--- lower = build(pts) -----------------------------------------------
--- upper = build(reversed(pts)) -------------------------------------
--- return lower[:-1] + upper[:-1] -----------------------------------
```

3.5.2. *Convex Hull 3D.*

```
class P3: --------------------------------------------------------------
--- __slots__ = ['x', 'y', 'z'] --------------------------------------
--- def __init__(self, x=0, y=0, z=0): -------------------------------
------- self.x, self.y, self.z = x, y, z ---------------------------
--- def __sub__(self, o): --------------------------------------------
------- return P3(self.x-o.x, self.y-o.y, self.z-o.z) --------------
--- def cross(self, o): ----------------------------------------------
------- return P3(self.y*o.z - self.z*o.y, -------------------------
----------------- self.z*o.x - self.x*o.z, -------------------------
----------------- self.x*o.y - self.y*o.x) -------------------------
--- def dot(self, o): ------------------------------------------------
------- return self.x*o.x + self.y*o.y + self.z*o.z ----------------
--- def __eq__(self, o): ---------------------------------------------
------- return (self.x, self.y, self.z) == (o.x, o.y, o.z) ---------
--- def __lt__(self, o): ---------------------------------------------
------- return (self.x, self.y, self.z) < (o.x, o.y, o.z) ----------
 ----------------------------------------------------------------------
class Face: ------------------------------------------------------------
--- __slots__ = ['p', 'q'] -------------------------------------------
--- def __init__(self, p, q): ----------------------------------------
------- self.p = p # Tuple of 3 point indices ----------------------
------- self.q = q # Normal vector (P3 object) ---------------------
 ----------------------------------------------------------------------
def hull_3D(pts): ------------------------------------------------------
--- n = len(pts) -----------------------------------------------------
--- faces = [] -------------------------------------------------------
--- dead = [[True] * n for _ in range(n)] ----------------------------
--- def add_face(a, b, c): -------------------------------------------
------- q = (pts[b] - pts[a]).cross(pts[c] - pts[a]) ---------------
------- faces.append(Face((a, b, c), q)) ---------------------------
------- dead[a][b] = dead[b][c] = dead[c][a] = False ---------------
--- # Initialize with a degenerate tetrahedron ----------------------
--- add_face(0, 1, 2) ------------------------------------------------
--- add_face(0, 2, 1) ------------------------------------------------
--- for i in range(3, n): --------------------------------------------
------- f_inv = [] -------------------------------------------------
------- for f in faces: --------------------------------------------
----------- # If face is visible from the point ---------------------
----------- if (pts[i] - pts[f.p[0]]).dot(f.q) > 0: ----------------
--------------- for j in range(3): -------------------------------
------------------- dead[f.p[j]][f.p[(j+1)%3]] = True --------------
----------- else: --------------------------------------------------
--------------- f_inv.append(f) ----------------------------------
------- faces = [] -------------------------------------------------
------- for f in f_inv: --------------------------------------------
----------- for j in range(3): -------------------------------------
--------------- a, b = f.p[j], f.p[(j+1)%3] ----------------------
--------------- # If reverse edge is dead but forward is alive -----
--------------- if dead[b][a]: -----------------------------------
------------------- add_face(b, a, i) --------------------------
------- faces.extend(f_inv) ----------------------------------------
--- return faces -----------------------------------------------------
```

3.6. Areas.

3.6.1. *Triangle Area.*

```
def area(a, b, c): -----------------------------------------------------
--- s = (a + b + c) / 2 ----------------------------------------------
--- return (s*(s-a)*(s-b)*(s-c)) ** 0.5 ------------------------------
```

3.6.2. *Polygon Area.*

```
def area(p, n): --------------------------------------------------------
--- a = 0 ------------------------------------------------------------
--- for i in range(n): -----------------------------------------------
------- j = (i - 1) % n --------------------------------------------
------- a += cross(p[i], p[j]) -------------------------------------
--- return abs(a) / 2 ------------------------------------------------
```

3.7. Dot Cross.

```
def dot(a, b): ---------------------------------------------------------
--- return a.x * b.x + a.y * b.y # + a.z * b.z -----------------------
def cross(a, b): -------------------------------------------------------
--- return a.x * b.y - a.y * b.x -------------------------------------
def cross(a, b, c): ----------------------------------------------------
--- return cross(a, b) + cross(b, c) + cross(c, a) -------------------
def cross3D(a, b): -----------------------------------------------------
--- return point3(a.x*b.y - a.y*b.x, a.y*b.z - a.z*b.y, a.z*b.x -
↪   a.x*b.z)
```

3.8. Line Sweep Closest Pair.

```
import math

def dist_sq(p1, p2):
    return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2


def closest_pair(points):
    points.sort()

    def solve(pts):
        n = len(pts)

        if n <= 3:
            return min(dist_sq(pts[i], pts[j])
                       for i in range(n)
                       for j in range(i + 1, n)) if n > 1 else
↪   float('inf')

        mid = n // 2
        mid_x = pts[mid][0]

        left = pts[:mid]
        right = pts[mid:]

        d = min(solve(left), solve(right))

        strip = [p for p in pts if (p[0] - mid_x) ** 2 < d]
        strip.sort(key=lambda p: p[1])

        for i in range(len(strip)):
            for j in range(i + 1, min(i + 7, len(strip))):
                d = min(d, dist_sq(strip[i], strip[j]))

        return d

    return solve(points)
```

3.9. Point Projection.

```
EPS = 1e-9

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))


def scale(v, k):
    return [k * x for x in v]


def add(a, b):
    return [x + y for x, y in zip(a, b)]


def sub(a, b):
    return [x - y for x, y in zip(a, b)]


def norm(v):
    return dot(v, v)

def proj(p, v):
    v_norm = norm(v)
    if v_norm < EPS:
        return [0] * len(p)
    return scale(v, dot(p, v) / v_norm)

def proj_line(p, a, b):
    ab = sub(b, a)
    ab_norm = norm(ab)
    if ab_norm < EPS:
        return a[:]

    t = dot(sub(p, a), ab) / ab_norm
    return add(a, scale(ab, t))

def proj_seg(p, a, b):
    ab = sub(b, a)
    ab_norm = norm(ab)
    if ab_norm < EPS:
        return a[:]

    t = dot(sub(p, a), ab) / ab_norm
    t = max(0.0, min(1.0, t))

    return add(a, scale(ab, t))

def proj_plane(p, a, b, c, d):
    n = [a, b, c]
    n_norm = norm(n)
    if n_norm < EPS:
        return p[:]

    dist = (dot(p, n) + d) / n_norm
    return sub(p, scale(n, dist))
```

3.10. Great Circle Distance.

```
import math

PI = math.pi

def clamp(x, lo, hi):
    return max(lo, min(hi, x))

# ========================================================
# version 1: lat/long (spherical law of cosines)
# ========================================================
def great_circle_dist_latlong(lat1, lon1, lat2, lon2, R=6371.0):
    lat1 *= PI / 180
    lon1 *= PI / 180
    lat2 *= PI / 180
    lon2 *= PI / 180

    val = (
        math.sin(lat1) * math.sin(lat2)
        + math.cos(lat1) * math.cos(lat2) * math.cos(abs(lon1 - lon2))
    )

    val = clamp(val, -1.0, 1.0)  # IMPORTANT FIX

    return R * math.acos(val)

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def cross(a, b):
    return [
        a[1] * b[2] - a[2] * b[1],
        a[2] * b[0] - a[0] * b[2],
        a[0] * b[1] - a[1] * b[0],
    ]

def norm(v):
    return math.sqrt(dot(v, v))

# ========================================================
# version 2: using 3D vectors (angle between vectors)
# ========================================================
def great_circle_dist_vec(a, b, R=1.0):
    # assumes a and b are unit vectors on sphere
    cross_prod = cross(a, b)
    return R * math.atan2(norm(cross_prod), dot(a, b))

def to_vec(lat, lon):
    lat *= PI / 180
    lon *= PI / 180

    return [
        math.cos(lat) * math.cos(lon),
        math.cos(lat) * math.sin(lon),
        math.sin(lat),
    ]
```

3.11. Point In Polygon.

```
import math

EPS = 1e-9

def dist(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])

def in_polygon(q, p):
    n = len(p)
    inside = False

    j = n - 1
    for i in range(n):
        xi, yi = p[i]
        xj, yj = p[j]
        xq, yq = q

        intersect = ((yi > yq) != (yj > yq)) and (
            xq < (xj - xi) * (yq - yi) / (yj - yi + EPS) + xi
        )

        if intersect:
            inside = not inside

        j = i

    return inside

def on_polygon(q, p):
    n = len(p)

    for i in range(n):
        j = (i + 1) % n

        a = p[i]
        b = p[j]

        if abs(dist(a, q) + dist(q, b) - dist(a, b)) < EPS:
            return True

    return False
```

3.12. Cut Polygon By Line.

```
EPS = 1e-9

def cross(a, b, c):
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] -
↪   a[0])

def line_inter(p1, p2, a, b):
    x1, y1 = p1
    x2, y2 = p2
    x3, y3 = a
    x4, y4 = b

    den = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)
    if abs(den) < EPS:
        return p1
```

```
--------------------------------------------------------------
--- px = (
------- (x1 * y2 - y1 * x2) * (x3 - x4)
------- - (x1 - x2) * (x3 * y4 - y3 * x4)
--- ) / den
--------------------------------------------------------------
--- py = (
------- (x1 * y2 - y1 * x2) * (y3 - y4)
------- - (y1 - y2) * (x3 * y4 - y3 * x4)
--- ) / den
--------------------------------------------------------------
--- return (px, py)
--------------------------------------------------------------
def cut(poly, a, b):
--- n = len(poly)
--- res = []
--------------------------------------------------------------
--- for i in range(n):
------- j = (i - 1 + n) % n
--------------------------------------------------------------
------- c1 = cross(a, b, poly[j])
------- c2 = cross(a, b, poly[i])
--------------------------------------------------------------
------- if c1 >= -EPS:
----------- res.append(poly[j])
--------------------------------------------------------------
------- if c1 * c2 < -EPS:
----------- inter = line_inter(poly[j], poly[i], a, b)
----------- res.append(inter)
--------------------------------------------------------------
--- return res
```

3.13. Triangle Centers.

```
import math
--------------------------------------------------------------
EPS = 1e-9
--------------------------------------------------------------
def add(a, b):
--- return (a[0] + b[0], a[1] + b[1])
def sub(a, b):
--- return (a[0] - b[0], a[1] - b[1])
def scale(a, k):
--- return (a[0] * k, a[1] * k)
def norm2(a, b):
--- return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
def bary(A, B, C, a, b, c):
--- s = a + b + c
--- if abs(s) < EPS:
------- return (0.0, 0.0)
--------------------------------------------------------------
--- return (
------- (A[0] * a + B[0] * b + C[0] * c) / s,
------- (A[1] * a + B[1] * b + C[1] * c) / s,
--- )
def dist(a, b):
--- return math.sqrt(norm2(a, b))
def trilinear(A, B, C, a, b, c):
--- return bary(
------- A,
------- B,
------- C,
------- dist(B, C) * a,
------- dist(C, A) * b,
------- dist(A, B) * c,
--- )
def centroid(A, B, C):
--- return bary(A, B, C, 1, 1, 1)
def circumcenter(A, B, C):
--- a = dist(B, C)
--- b = dist(C, A)
--- c = dist(A, B)
--------------------------------------------------------------
--- return bary(
------- A,
------- B,
------- C,
------- a * (b + c - a),
------- b * (c + a - b),
------- c * (a + b - c),
--- )
def orthocenter(A, B, C):
--- def foot(P, Q, R):
------- q = sub(Q, R)
------- t = dot(sub(P, R), q) / (dot(q, q) + EPS)
------- return add(R, scale(q, t))
--- def dot(a, b):
------- return a[0] * b[0] + a[1] * b[1]
--------------------------------------------------------------
--- H1 = foot(A, B, C)
--- H2 = foot(B, A, C)
--------------------------------------------------------------
--- return bary(A, B, C, 1, 1, 1)
def incenter(A, B, C):
--- return bary(
------- A,
------- B,
------- C,
------- dist(B, C),
------- dist(C, A),
------- dist(A, B),
--- )
def inradius(a, b, c):
--- s = (a + b + c) / 2
--- area = math.sqrt(max(s * (s - a) * (s - b) * (s - c), 0))
--- return area / s if s > EPS else 0
def excenter(A, B, C):
--- a = dist(B, C)
--- b = dist(C, A)
--- c = dist(A, B)
--- return bary(A, B, C, -a, b, c)
def symmedian(A, B, C):
--- return bary(
------- A,
------- B,
------- C,
------- norm2(B, C),
------- norm2(C, A),
------- norm2(A, B),
--- )
```

3.14. Convex Polygon Intersection.

```
EPS = 1e-9
--------------------------------------------------------------
def sub(a, b):
--- return (a[0] - b[0], a[1] - b[1])
def add(a, b):
--- return (a[0] + b[0], a[1] + b[1])
def cross(a, b):
--- return a[0] * b[1] - a[1] * b[0]
def dot(a, b):
--- return a[0] * b[0] + a[1] * b[1]
def orient(a, b, c):
--- return cross(sub(b, a), sub(c, a))
def seg_inter(a, b, c, d):
--- def on_seg(p, a, b):
------- return (
----------- abs(orient(a, b, p)) < EPS
----------- and min(a[0], b[0]) - EPS <= p[0] <= max(a[0], b[0]) + EPS
----------- and min(a[1], b[1]) - EPS <= p[1] <= max(a[1], b[1]) + EPS
------- )
--- def line_inter(a, b, c, d):
------- A1 = b[1] - a[1]
------- B1 = a[0] - b[0]
------- C1 = A1 * a[0] + B1 * a[1]
------- A2 = d[1] - c[1]
------- B2 = c[0] - d[0]
------- C2 = A2 * c[0] + B2 * c[1]
------- det = A1 * B2 - A2 * B1
------- if abs(det) < EPS:
----------- return None
------- x = (B2 * C1 - B1 * C2) / det
------- y = (A1 * C2 - A2 * C1) / det
------- return (x, y)
--- p = line_inter(a, b, c, d)
--- if p is None:
------- return None
--- if (
------- on_seg(p, a, b)
------- and on_seg(p, c, d)
--- ):
------- return p
--- return None
def in_poly(p, poly):
--- inside = False
--- n = len(poly)
--- for i in range(n):
------- j = (i - 1) % n
------- a, b = poly[i], poly[j]
------- if ((a[1] > p[1]) != (b[1] > p[1])):
----------- x = (b[0] - a[0]) * (p[1] - a[1]) / (b[1] - a[1] + EPS) +
↪  a[0]
----------- if p[0] < x:
--------------- inside = not inside
--- return inside
def on_poly(p, poly):
--- n = len(poly)
--- for i in range(n):
------- j = (i - 1) % n
------- if abs(orient(poly[j], poly[i], p)) < EPS:
----------- return True
--- return False
def convex_hull(points):
--- points = sorted(points)
--- def build(h):
------- for p in points:
----------- while len(h) >= 2 and orient(h[-2], h[-1], p) <= EPS:
--------------- h.pop()
----------- h.append(p)
------- return h
--- lower = build([])
--- upper = build([])
--- return lower[:-1] + upper[:-1]
def convex_polygon_inter(a, b):
--- ans = []
--- for p in a:
------- if in_poly(p, b) or on_poly(p, b):
----------- ans.append(p)
--- for p in b:
------- if in_poly(p, a) or on_poly(p, a):
----------- ans.append(p)
--- for i in range(len(a)):
------- I = (i - 1) % len(a)
------- for j in range(len(b)):
----------- J = (j - 1) % len(b)
----------- p = seg_inter(a[I], a[i], b[J], b[j])
----------- if p is not None:
--------------- ans.append(p)
--- if not ans:
------- return []
--- return convex_hull(ans)
```

3.15. Pick Theorem Lattice.

```python
import math

def area(poly):
    n = len(poly)
    s = 0
    for i in range(n):
        j = (i - 1) % n
        s += poly[j][0] * poly[i][1] - poly[i][0] * poly[j][1]
    return abs(s) // 2
def boundary(poly):
    n = len(poly)
    ans = 0
    for i in range(n):
        j = (i - 1) % n
        dx = poly[i][0] - poly[j][0]
        dy = poly[i][1] - poly[j][1]
        ans += math.gcd(abs(dx), abs(dy))
    return ans
def interior(poly):
    A = area(poly)
    B = boundary(poly)
    return A - B // 2 + 1
```

3.16. Minimum Enclosing Circle.

```python
import math

EPS = 1e-9
def dist(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])
def mid(a, b):
    return ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
def cross(a, b, c):
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] -
↪   a[0])
def circumcenter(a, b, c):
    ax, ay = a
    bx, by = b
    cx, cy = c
    d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
    if abs(d) < EPS:
        return mid(a, b)
    ux = (
        (ax * ax + ay * ay) * (by - cy)
        + (bx * bx + by * by) * (cy - ay)
        + (cx * cx + cy * cy) * (ay - by)
    ) / d
    uy = (
        (ax * ax + ay * ay) * (cx - bx)
        + (bx * bx + by * by) * (ax - cx)
        + (cx * cx + cy * cy) * (bx - ax)
    ) / d
    return (ux, uy)
def bounding_ball(p):
    random.shuffle(p)
    center = p[0]
    radius = 0.0
    n = len(p)
    for i in range(n):
        if dist(center, p[i]) > radius + EPS:
            center = p[i]
            radius = 0.0
            for j in range(i):
                if dist(center, p[j]) > radius + EPS:
                    center = mid(p[i], p[j])
                    radius = dist(center, p[i])
                    for k in range(j):
                        if dist(center, p[k]) > radius + EPS:
                            center = circumcenter(p[i], p[j], p[k])
                            radius = dist(center, p[i])
    return center, radius
def verify(p, center, r):
    for x in p:
        if dist(x, center) > r + 1e-6:
            return False
    return True
```

3.17. Shamos Algorithm.

```python
import math

EPS = 1e-9
def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])
def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]
def dist_pt_line(p, a, b):
    ab = sub(b, a)
    ap = sub(p, a)
    return abs(cross(ab, ap)) / (math.hypot(ab[0], ab[1]) + EPS)
def convex_hull(points):
    points = sorted(points)
    def build():
        hull = []
        for p in points:
            while len(hull) >= 2 and cross(sub(hull[-1], hull[-2]),
↪   sub(p, hull[-1])) <= EPS:
                hull.pop()
            hull.append(p)
        return hull
    lower = build()
    upper = build()
    return lower[:-1] + upper[:-1]
def shamos(p):
    h = convex_hull(p)
    k = len(h)

    if k <= 2:
        return 0.0

    d = float("inf")
    j = 1

    for i in range(k):
        ni = (i + 1) % k

        a = h[i]
        b = h[ni]

        while True:
            nj = (j + 1) % k
            if dist_pt_line(h[nj], a, b) > dist_pt_line(h[j], a, b):
                j = nj
            else:
                break
        d = min(d, dist_pt_line(h[j], a, b))
    return d
```

3.18. Kd Tree.

```python
import heapq

EPS = 1e-9
def dist2(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
class KDTree:
    def __init__(self, points):
        self.p = points[:]
        self.n = len(points)
        self.build(0, self.n, 0)
    def build(self, L, R, dvx):
        if R - L <= 1:
            return
        mid = (L + R) // 2
        if dvx:
            self.p[L:R] = sorted(self.p[L:R], key=lambda x: x[0])
        else:
            self.p[L:R] = sorted(self.p[L:R], key=lambda x: x[1])
        self.build(L, mid, not dvx)
        self.build(mid + 1, R, not dvx)
    def knn(self, x, y, k=1, r=-1):
        self.qx = x
        self.qy = y
        self.k = k
        self.prune = float("inf") if r < 0 else r * r
        self.heap = []
        self._dfs(0, self.n, False)
        res = [p for _, p in sorted(self.heap, reverse=True)]
        return res
    def _dfs(self, L, R, dvx):
        if R - L <= 0:
            return
        mid = (L + R) // 2
        p = self.p[mid]
        dx = self.qx - p[0]
        dy = self.qy - p[1]
        D = dx * dx + dy * dy
        if D <= self.prune:
            heapq.heappush(self.heap, (-D, p))
            if len(self.heap) > self.k:
                heapq.heappop(self.heap)
        delta = dx if dvx else dy
        nL, nR = L, mid
        fL, fR = mid + 1, R
        if delta > 0:
            nL, fL = fL, nL
            nR, fR = fR, nR
        self._dfs(nL, nR, not dvx)
        if len(self.heap) < self.k or delta * delta <= self.prune:
            self._dfs(fL, fR, not dvx)
```

4. Graphs

4.1. Max Flow.

4.1.1. *Dinic.*

```python
INF = float('inf')
class Dinic:
    def __init__(self, n, s, t):
        self.n, self.s, self.t = n, s, t
        self.edges = []
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v, c, bi=False):
        self.adj[u].append(len(self.edges))
        self.edges.append([v, c, 0])
        self.adj[v].append(len(self.edges))
        self.edges.append([u, c if bi else 0, 0])

    def bfs(self):
        self.dist = [-1] * self.n
        self.dist[self.s] = 0
        q = deque([self.s])
        while q:
            u = q.popleft()
            for i in self.adj[u]:
                v, c, f = self.edges[i]
                if self.dist[v] < 0 and c > f:
                    self.dist[v] = self.dist[u] + 1
                    q.append(v)
        return self.dist[self.t] != -1

    def dfs(self, u, pushed):
        if pushed == 0 or u == self.t: return pushed
        while self.ptr[u] < len(self.adj[u]):
```

```
----------- i = self.adj[u][self.ptr[u]] ----------------------------
----------- v, c, f = self.edges[i] ---------------------------------
----------- if self.dist[u]+1 == self.dist[v] and c > f: -------------
--------------- tr = self.dfs(v, min(pushed, c - f)) -----------------
--------------- if tr > 0: --------------------------------------------
------------------- self.edges[i][2] += tr --------------------------
------------------- self.edges[i^1][2] -= tr ------------------------
------------------- return tr ---------------------------------------
----------- self.ptr[u] += 1 ----------------------------------------
------- return 0 -----------------------------------------------------

--- def calc_max_flow(self): -------------------------------------------
------- flow = 0 ----------------------------------------------------
------- while self.bfs(): -------------------------------------------
----------- self.ptr = [0] * self.n ---------------------------------
----------- while True: ---------------------------------------------
--------------- pushed = self.dfs(self.s, INF) ----------------------
--------------- if not pushed: --------------------------------------
------------------- break -------------------------------------------
--------------- flow += pushed --------------------------------------
------- return flow --------------------------------------------------
```

4.2. Shortest Path.

4.2.1. *Dijkstra.*

```
import heapq -------------------------------------------------------
INF = float('inf') -------------------------------------------------
def dijkstra(n, adj, s): -------------------------------------------
--- dist = [INF] * n -----------------------------------------------
--- dist[s] = 0 ----------------------------------------------------
--- pq = [(0, s)] --------------------------------------------------
--- ---------------------------------------------------------------
--- while pq: ------------------------------------------------------
------- d, u = heapq.heappop(pq) -------------------------------------
------- if dist[u] < d: ----------------------------------------------
----------- continue ------------------------------------------------
------- for v, w in adj[u]: ------------------------------------------
----------- if dist[v] > d + w: -------------------------------------
--------------- dist[v] = d + w -------------------------------------
--------------- heapq.heappush(pq, (dist[v], v)) --------------------
--- return dist ----------------------------------------------------
```

4.2.2. *Bellman Ford.*

```
INF = float('inf') -------------------------------------------------
def bellman_ford(n, adj, s): ---------------------------------------
--- dist = [INF] * n -----------------------------------------------
--- dist[s] = 0 ----------------------------------------------------
--- for _ in range(n - 1): -----------------------------------------
------- for u in range(n): -------------------------------------------
----------- if dist[u] == INF: continue -----------------------------
----------- for v, w in adj[u]: -------------------------------------
--------------- if dist[u] + w < dist[v]: ---------------------------
------------------- dist[v] = dist[u] + w -------------------------
--- for u in range(n): ---------------------------------------------
------- if dist[u] == INF: continue ----------------------------------
------- for v, w in adj[u]: ------------------------------------------
----------- if dist[u] + w < dist[v]: -------------------------------
--------------- return dist, True -----------------------------------
--- return dist, False ---------------------------------------------
```

4.3. Minimum Spanning Tree.

4.3.1. *Kruskal Min Spanning Tree.*

```
import heapq -------------------------------------------------------
---------------------------------------------------------------------

class UnionFind: ---------------------------------------------------
--- def __init__(self, n): -----------------------------------------
------- self.parent = list(range(n)) ---------------------------------
------- self.rank = [0] * n ------------------------------------------
---------------------------------------------------------------------
--- def find(self, x): ---------------------------------------------
------- while self.parent[x] != x: -----------------------------------
----------- self.parent[x] = self.parent[self.parent[x]] ------------
----------- x = self.parent[x] --------------------------------------
------- return x -----------------------------------------------------
---------------------------------------------------------------------
--- def unite(self, a, b): -----------------------------------------
------- a = self.find(a) ---------------------------------------------
------- b = self.find(b) ---------------------------------------------
---------------------------------------------------------------------
------- if a == b: ---------------------------------------------------
----------- return False --------------------------------------------
---------------------------------------------------------------------
------- if self.rank[a] < self.rank[b]: ------------------------------
----------- self.parent[a] = b --------------------------------------
------- else: --------------------------------------------------------
----------- self.parent[b] = a --------------------------------------
----------- if self.rank[a] == self.rank[b]: ------------------------
--------------- self.rank[a] += 1 -----------------------------------
---------------------------------------------------------------------
------- return True --------------------------------------------------
---------------------------------------------------------------------
---------------------------------------------------------------------
def kruskal(n, edges): ---------------------------------------------
--- edges = edges[:] -----------------------------------------------
--- heapq.heapify(edges) -------------------------------------------
---------------------------------------------------------------------
--- uf = UnionFind(n) ----------------------------------------------
--- total_weight = 0 -----------------------------------------------
--- mst = [] -------------------------------------------------------
---------------------------------------------------------------------
--- while edges: ---------------------------------------------------
------- w, u, v = heapq.heappop(edges) -------------------------------
------- if uf.unite(u, v): -------------------------------------------
----------- mst.append((u, v, w)) -----------------------------------
----------- total_weight += w ---------------------------------------
---------------------------------------------------------------------
----------- if len(mst) == n - 1: -----------------------------------
--------------- break -----------------------------------------------
---------------------------------------------------------------------
--- return mst, total_weight ---------------------------------------
```

4.3.2. *Prim Min Spanning Tree.*

```
import heapq -------------------------------------------------------
---------------------------------------------------------------------
def prim(n, adj, start=0): -----------------------------------------
--- visited = [False] * n ------------------------------------------
--- pq = [(0, start, -1)] ------------------------------------------
---------------------------------------------------------------------
--- total_weight = 0 -----------------------------------------------
--- mst = [] -------------------------------------------------------
---------------------------------------------------------------------
--- while pq: ------------------------------------------------------
------- w, u, parent = heapq.heappop(pq) -----------------------------
---------------------------------------------------------------------
------- if visited[u]: -----------------------------------------------
----------- continue ------------------------------------------------
---------------------------------------------------------------------
------- visited[u] = True --------------------------------------------
---------------------------------------------------------------------
------- if parent != -1: ---------------------------------------------
----------- mst.append((parent, u, w)) ------------------------------
----------- total_weight += w ---------------------------------------
---------------------------------------------------------------------
------- for v, weight in adj[u]: -------------------------------------
----------- if not visited[v]: --------------------------------------
--------------- heapq.heappush(pq, (weight, v, u)) ------------------
---------------------------------------------------------------------
--- return mst, total_weight ---------------------------------------
```

4.4. Least Common Ancestor.

4.4.1. *Binary Lifting Lca.*

```
import math ---------------------------------------------------------
---------------------------------------------------------------------
class BinaryLifting: -----------------------------------------------
--- def __init__(self, adj): ---------------------------------------
------- self.n = len(adj) --------------------------------------------
------- self.LOG = math.ceil(math.log2(self.n)) + 1 ------------------
------- self.adj = adj -----------------------------------------------
---------------------------------------------------------------------
------- self.dep = [0] * self.n --------------------------------------
------- self.par = [[-1] * self.LOG for _ in range(self.n)] ----------
---------------------------------------------------------------------
------- self.build(0) ------------------------------------------------
---------------------------------------------------------------------
--- def dfs(self, u, p): -------------------------------------------
------- self.par[u][0] = p -------------------------------------------
------- for v in self.adj[u]: ----------------------------------------
----------- if v != p: ----------------------------------------------
--------------- self.dep[v] = self.dep[u] + 1 -----------------------
--------------- self.dfs(v, u) --------------------------------------
---------------------------------------------------------------------
--- def build(self, root): -----------------------------------------
------- self.dfs(root, root) -----------------------------------------
---------------------------------------------------------------------
------- for k in range(1, self.LOG): ---------------------------------
----------- for u in range(self.n): ---------------------------------
--------------- self.par[u][k] = self.par[self.par[u][k-1]][k-1] ----
---------------------------------------------------------------------
--- def lift(self, u, k): ------------------------------------------
------- for i in range(self.LOG): ------------------------------------
----------- if k & (1 << i): ----------------------------------------
--------------- u = self.par[u][i] ----------------------------------
------- return u -----------------------------------------------------
---------------------------------------------------------------------
--- def lca(self, u, v): -------------------------------------------
------- if self.dep[u] < self.dep[v]: --------------------------------
----------- u, v = v, u ---------------------------------------------
---------------------------------------------------------------------
------- u = self.lift(u, self.dep[u] - self.dep[v]) ------------------
---------------------------------------------------------------------
------- if u == v: ---------------------------------------------------
----------- return u ------------------------------------------------
---------------------------------------------------------------------
------- for k in reversed(range(self.LOG)): --------------------------
----------- if self.par[u][k] != self.par[v][k]: --------------------
--------------- u = self.par[u][k] ----------------------------------
--------------- v = self.par[v][k] ----------------------------------
---------------------------------------------------------------------
------- return self.par[u][0] ----------------------------------------
```

4.4.2. *Euler Lca.*

```
class EulerLCA: ----------------------------------------------------
--- def __init__(self, adj): ---------------------------------------
------- self.n = len(adj) --------------------------------------------
------- self.adj = adj -----------------------------------------------
---------------------------------------------------------------------
------- self.euler = [] ----------------------------------------------
------- self.first = [-1] * self.n -----------------------------------
------- self.depth = [0] * self.n ------------------------------------
---------------------------------------------------------------------
------- self.build(0) ------------------------------------------------
---------------------------------------------------------------------
--- def dfs(self, u, p, d): ----------------------------------------
------- self.first[u] = len(self.euler) ------------------------------
------- self.depth[u] = d --------------------------------------------
------- self.euler.append(u) -----------------------------------------
---------------------------------------------------------------------
------- for v in self.adj[u]: ----------------------------------------
----------- if v != p: ----------------------------------------------
```

```
---------------- self.dfs(v, u, d + 1) ------------------------------
---------------- self.euler.append(u) -------------------------------
-------------------------------------------------------------------
--- def build(self, root): ----------------------------------------
------- self.dfs(root, root, 0) -----------------------------------
-------------------------------------------------------------------
------- m = len(self.euler) ---------------------------------------
------- LOG = m.bit_length() --------------------------------------
-------------------------------------------------------------------
------- self.st = [[0] * m for _ in range(LOG)] -------------------
------- self.log = [0] * (m + 1) ----------------------------------
-------------------------------------------------------------------
------- for i in range(2, m + 1): ---------------------------------
----------- self.log[i] = self.log[i // 2] + 1 ------------------
-------------------------------------------------------------------
------- for i in range(m): ----------------------------------------
----------- self.st[0][i] = self.euler[i] -----------------------
-------------------------------------------------------------------
------- for k in range(1, LOG): -----------------------------------
----------- for i in range(m - (1 << k) + 1): -------------------
--------------- a = self.st[k-1][i] -------------------------------
--------------- b = self.st[k-1][i + (1 << (k-1))] ----------------
--------------- self.st[k][i] = a if self.depth[a] < self.depth[b] else
↪  b
-------------------------------------------------------------------
--- def lca(self, u, v): ------------------------------------------
------- l, r = self.first[u], self.first[v] -----------------------
------- if l > r: -------------------------------------------------
----------- l, r = r, l -----------------------------------------
-------------------------------------------------------------------
------- k = self.log[r - l + 1] -----------------------------------
------- a = self.st[k][l] -----------------------------------------
------- b = self.st[k][r - (1 << k) + 1] --------------------------
-------------------------------------------------------------------
------- return a if self.depth[a] < self.depth[b] else b ----------
```

4.4.3. *Tarjan Lca.*

```
class DSU: ----------------------------------------------------------
--- def __init__(self, n): ----------------------------------------
------- self.p = list(range(n)) -----------------------------------
-------------------------------------------------------------------
--- def find(self, x): --------------------------------------------
------- if self.p[x] != x: ----------------------------------------
----------- self.p[x] = self.find(self.p[x]) --------------------
------- return self.p[x] ------------------------------------------
-------------------------------------------------------------------
--- def unite(self, a, b): ----------------------------------------
------- self.p[self.find(b)] = self.find(a) -----------------------
-------------------------------------------------------------------
-------------------------------------------------------------------
class Tarjan: -------------------------------------------------------
--- def __init__(self, adj, queries): -----------------------------
------- self.adj = adj --------------------------------------------
------- self.queries = queries ------------------------------------
------- self.n = len(adj) -----------------------------------------
-------------------------------------------------------------------
------- self.dsu = DSU(self.n) ------------------------------------
------- self.vis = [False] * self.n -------------------------------
------- self.anc = [0] * self.n -----------------------------------
------- self.res = {} ---------------------------------------------
-------------------------------------------------------------------
--- def dfs(self, u, p): ------------------------------------------
------- self.anc[u] = u -------------------------------------------
-------------------------------------------------------------------
------- for v in self.adj[u]: -------------------------------------
----------- if v == p: ------------------------------------------
--------------- continue ----------------------------------------
----------- self.dfs(v, u) --------------------------------------
----------- self.dsu.unite(u, v) --------------------------------
----------- self.anc[self.dsu.find(u)] = u ----------------------
-------------------------------------------------------------------
------- self.vis[u] = True ----------------------------------------
-------------------------------------------------------------------
------- for v, idx in self.queries[u]: ----------------------------
----------- if self.vis[v]: -------------------------------------
--------------- self.res[idx] = self.anc[self.dsu.find(v)] ------
```

4.5. Minimum Mean Weight Cycle.

```
def min_mean_cycle(adj): --------------------------------------------
--- n = len(adj) --------------------------------------------------
--- INF = float('inf') --------------------------------------------
-------------------------------------------------------------------
--- arr = [[INF] * n for _ in range(n + 1)] -----------------------
-------------------------------------------------------------------
--- for i in range(n): --------------------------------------------
------- arr[0][i] = 0 ---------------------------------------------
-------------------------------------------------------------------
--- for k in range(1, n + 1): -------------------------------------
------- for u in range(n): ----------------------------------------
----------- if arr[k - 1][u] == INF: ----------------------------
--------------- continue ----------------------------------------
----------- for v, w in adj[u]: ---------------------------------
--------------- arr[k][v] = min(arr[k][v], arr[k - 1][u] + w) ---
-------------------------------------------------------------------
--- result = INF --------------------------------------------------
-------------------------------------------------------------------
--- for v in range(n): --------------------------------------------
------- if arr[n][v] == INF: --------------------------------------
----------- continue --------------------------------------------
-------------------------------------------------------------------
------- max_avg = -INF --------------------------------------------
------- for k in range(n): ----------------------------------------
----------- if arr[k][v] == INF: --------------------------------
--------------- continue ----------------------------------------
----------- avg = (arr[n][v] - arr[k][v]) / (n - k) -------------
----------- max_avg = max(max_avg, avg) -------------------------
-------------------------------------------------------------------
------- result = min(result, max_avg) -----------------------------
-------------------------------------------------------------------
--- return result -------------------------------------------------
```

4.6. Strongly Connected Components.

```
class KosarajuGraph: ------------------------------------------------
--- def __init__(self, n): ----------------------------------------
------- self.n = n ------------------------------------------------
------- self.graph = [[] for _ in range(n)] -----------------------
------- self.rev_graph = [[] for _ in range(n)] -------------------
------- self.vis = [0] * n ----------------------------------------
------- self.sccs = [] --------------------------------------------
-------------------------------------------------------------------
--- def add_edge(self, u, v): -------------------------------------
------- self.graph[u].append(v) -----------------------------------
------- self.rev_graph[v].append(u) -------------------------------
-------------------------------------------------------------------
--- def dfs(self, u, adj, container): -----------------------------
------- self.vis[u] = 1 -------------------------------------------
------- for v in adj[u]: ------------------------------------------
----------- if not self.vis[v]: ---------------------------------
--------------- self.dfs(v, adj, container) ---------------------
------- container.append(u) ---------------------------------------
-------------------------------------------------------------------
--- def kosaraju(self): -------------------------------------------
------- topo = [] -------------------------------------------------
-------------------------------------------------------------------
------- # First pass ---------------------------------------------
------- self.vis = [0] * self.n -----------------------------------
------- for i in range(self.n): -----------------------------------
----------- if not self.vis[i]: ---------------------------------
--------------- self.dfs(i, self.graph, topo) -------------------
-------------------------------------------------------------------
------- # Second pass --------------------------------------------
------- self.vis = [0] * self.n -----------------------------------
------- self.sccs = [] --------------------------------------------
-------------------------------------------------------------------
------- for u in reversed(topo): ----------------------------------
----------- if not self.vis[u]: ---------------------------------
--------------- comp = [] ---------------------------------------
--------------- self.dfs(u, self.rev_graph, comp) ---------------
--------------- self.sccs.append(comp) --------------------------
-------------------------------------------------------------------
------- return self.sccs ------------------------------------------
```

4.7. Heavy Light Decomposition.

```
class SegmentTree: --------------------------------------------------
--- def __init__(self, n): ----------------------------------------
------- self.n = n ------------------------------------------------
------- self.tree = [0] * (4 * n) ---------------------------------
-------------------------------------------------------------------
--- # Example: point update -----------------------------------------
--- def update(self, idx, val, node=1, l=0, r=None): --------------
------- if r is None: ---------------------------------------------
----------- r = self.n - 1 --------------------------------------
------- if l == r: ------------------------------------------------
----------- self.tree[node] = vala ------------------------------
----------- return ----------------------------------------------
------- mid = (l + r) // 2 ----------------------------------------
------- if idx <= mid: --------------------------------------------
----------- self.update(idx, val, node * 2, l, mid) -------------
------- else: -----------------------------------------------------
----------- self.update(idx, val, node * 2 + 1, mid + 1, r) -----
------- self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]
-------------------------------------------------------------------
--- # Example: range query ------------------------------------------
--- def query(self, ql, qr, node=1, l=0, r=None): -----------------
------- if r is None: ---------------------------------------------
----------- r = self.n - 1 --------------------------------------
------- if qr < l or r < ql: --------------------------------------
----------- return 0 --------------------------------------------
------- if ql <= l and r <= qr: -----------------------------------
----------- return self.tree[node] ------------------------------
------- mid = (l + r) // 2 ----------------------------------------
------- return self.query(ql, qr, node * 2, l, mid) + \ ----------
--------------- self.query(ql, qr, node * 2 + 1, mid + 1, r) -----
-------------------------------------------------------------------
-------------------------------------------------------------------
class HeavyLightTree: -----------------------------------------------
--- def __init__(self, n): ----------------------------------------
------- self.n = n ------------------------------------------------
------- self.adj = [[] for _ in range(n)] -------------------------
-------------------------------------------------------------------
------- self.par = [-1] * n ---------------------------------------
------- self.heavy = [-1] * n -------------------------------------
------- self.dep = [0] * n ----------------------------------------
------- self.path_root = [0] * n ----------------------------------
------- self.pos = [0] * n ----------------------------------------
-------------------------------------------------------------------
------- self.segment_tree = SegmentTree(n) ------------------------
------- self.current_pos = 0 --------------------------------------
-------------------------------------------------------------------
--- def add_edge(self, u, v): -------------------------------------
------- self.adj[u].append(v) -------------------------------------
------- self.adj[v].append(u) -------------------------------------
-------------------------------------------------------------------
--- def dfs(self, u): ---------------------------------------------
------- size = 1 --------------------------------------------------
------- max_subtree_size = 0 --------------------------------------
-------------------------------------------------------------------
------- for v in self.adj[u]: -------------------------------------
----------- if v != self.par[u]: --------------------------------
--------------- self.par[v] = u ---------------------------------
--------------- self.dep[v] = self.dep[u] + 1 -------------------
--------------- subtree_size = self.dfs(v) ----------------------
-------------------------------------------------------------------
```

```
---------------- if subtree_size > max_subtree_size:
-------------------- max_subtree_size = subtree_size
-------------------- self.heavy[u] = v

---------------- size += subtree_size

------- return size

--- def decompose(self, u, root):
------- self.path_root[u] = root
------- self.pos[u] = self.current_pos
------- self.current_pos += 1

------- if self.heavy[u] != -1:
----------- self.decompose(self.heavy[u], root)

------- for v in self.adj[u]:
----------- if v != self.par[u] and v != self.heavy[u]:
---------------- self.decompose(v, v)

--- def build(self, root=0):
------- self.par[root] = root
------- self.dep[root] = 0
------- self.dfs(root)
------- self.decompose(root, root)

--- # Query path u -> v
--- def query(self, u, v):
------- res = 0
------- while self.path_root[u] != self.path_root[v]:
----------- if self.dep[self.path_root[u]] <
↪  self.dep[self.path_root[v]]:
---------------- u, v = v, u
----------- start = self.path_root[u]
----------- res += self.segment_tree.query(self.pos[start],
↪  self.pos[u])
----------- u = self.par[start]

------- # Same path
------- if self.dep[u] > self.dep[v]:
----------- u, v = v, u
------- res += self.segment_tree.query(self.pos[u], self.pos[v])
------- return res

--- # Update node value
--- def update(self, u, value):
------- self.segment_tree.update(self.pos[u], value)
```

4.8. Euler Path/Cycle.

4.8.1. *Directed Euler.*

```
from collections import defaultdict

class DirectedEuler:
--- def __init__(self, n):
------- self.n = n
------- self.adj = defaultdict(list)
------- self.indeg = [0]*n
------- self.outdeg = [0]*n
------- self.edge_count = 0

--- def add_edge(self, u, v):
------- self.adj[u].append(v)
------- self.outdeg[u] += 1
------- self.indeg[v] += 1
------- self.edge_count += 1

--- def find_start_node(self):
------- start = -1
------- end = -1
------- for i in range(self.n):
----------- if self.outdeg[i] - self.indeg[i] == 1:
---------------- if start != -1:
-------------------- return -1
---------------- start = i
----------- elif self.indeg[i] - self.outdeg[i] == 1:
---------------- if end != -1:
-------------------- return -1
---------------- end = i
----------- elif self.indeg[i] != self.outdeg[i]:
---------------- return -1

------- if start == -1:
----------- for i in range(self.n):
---------------- if self.outdeg[i] > 0:
-------------------- return i
------- return start

--- def get_euler_path(self):
------- start = self.find_start_node()
------- if start == -1:
----------- return None

------- stack = [start]
------- path = []

------- local_adj = {u: self.adj[u][:] for u in self.adj}

------- while stack:
----------- u = stack[-1]
----------- if u in local_adj and local_adj[u]:
---------------- v = local_adj[u].pop()
---------------- stack.append(v)
----------- else:
---------------- path.append(stack.pop())

------- if len(path) != self.edge_count + 1:
----------- return None

------- return path[::-1]


--- class UndirectedEuler:
------- def __init__(self, n):
----------- self.n = n
----------- self.adj = defaultdict(list)
----------- self.edge_count = 0

------- def add_edge(self, u, v):
----------- self.adj[u].append(v)
----------- self.adj[v].append(u)
----------- self.edge_count += 1

------- def is_valid(self):
----------- odd = 0
----------- for i in range(self.n):
---------------- if len(self.adj[i]) % 2 != 0:
-------------------- odd += 1
----------- return odd == 0 or odd == 2

------- def get_euler_path(self):
----------- if not self.is_valid():
---------------- return None

----------- start = 0
----------- for i in range(self.n):
---------------- if len(self.adj[i]) % 2 == 1:
-------------------- start = i
-------------------- break
---------------- if len(self.adj[i]) > 0:
-------------------- start = i

----------- stack = [start]
----------- path = []

----------- local_adj = {u: self.adj[u][:] for u in self.adj}

----------- while stack:
---------------- u = stack[-1]
---------------- if u in local_adj and local_adj[u]:
-------------------- v = local_adj[u].pop()
-------------------- local_adj[v].remove(u)
-------------------- stack.append(v)
---------------- else:
-------------------- path.append(stack.pop())

----------- if len(path) != self.edge_count + 1:
---------------- return None

----------- return path[::-1]
```

4.8.2. *Undirected Euler.*

```
from collections import defaultdict

class UndirectedEuler:
--- def __init__(self, n):
------- self.n = n
------- self.adj = defaultdict(list)
------- self.edge_count = 0

--- def add_edge(self, u, v):
------- self.adj[u].append(v)
------- self.adj[v].append(u)
------- self.edge_count += 1

--- def is_valid(self):
------- odd = 0
------- for i in range(self.n):
----------- if len(self.adj[i]) % 2 != 0:
---------------- odd += 1
------- return odd == 0 or odd == 2

--- def get_euler_path(self):
------- if not self.is_valid():
----------- return None

------- start = 0
------- for i in range(self.n):
----------- if len(self.adj[i]) % 2 == 1:
---------------- start = i
---------------- break
----------- if len(self.adj[i]) > 0:
---------------- start = i

------- stack = [start]
------- path = []

------- local_adj = {u: self.adj[u][:] for u in self.adj}

------- while stack:
----------- u = stack[-1]
----------- if u in local_adj and local_adj[u]:
---------------- v = local_adj[u].pop()
---------------- local_adj[v].remove(u)
---------------- stack.append(v)
----------- else:
---------------- path.append(stack.pop())

------- if len(path) != self.edge_count + 1:
----------- return None

------- return path[::-1]
```

4.9. Tree Isomorphism.

```
from collections import deque
---------------------------------------------------------------------------
def bfs(u, adj, n):
--- vis = [False] * n
--- pre = [-1] * n
--- q = deque()
---------------------------------------------------------------------------
--- q.append(u)
--- vis[u] = True
---------------------------------------------------------------------------
--- last = u
---------------------------------------------------------------------------
--- while q:
------- u = q.popleft()
------- last = u
---------------------------------------------------------------------------
------- for v in adj[u]:
----------- if not vis[v]:
--------------- vis[v] = True
--------------- pre[v] = u
--------------- q.append(v)
---------------------------------------------------------------------------
--- return last, pre
---------------------------------------------------------------------------
def tree_centers(r, adj, n):
--- u, _ = bfs(r, adj, n)
--- v, pre = bfs(u, adj, n)
---------------------------------------------------------------------------
--- path = []
--- while v != -1:
------- path.append(v)
------- v = pre[v]
---------------------------------------------------------------------------
--- size = len(path)
---------------------------------------------------------------------------
--- med = [path[size // 2]]
--- if size % 2 == 0:
------- med.append(path[size // 2 - 1])
---------------------------------------------------------------------------
--- return med
---------------------------------------------------------------------------
def rootcode(u, adj, p, d, pr):
--- children = []
--- for v in adj[u]:
------- if v != p:
----------- children.append(rootcode(v, adj, u, (d + 1) % len(pr), pr))
---------------------------------------------------------------------------
--- children.sort()
---------------------------------------------------------------------------
--- h = len(children) + 1
--- for x in children:
------- h = h * pr[d] + x
---------------------------------------------------------------------------
--- return h
---------------------------------------------------------------------------
def treecode(root, adj, n, pr):
--- c = tree_centers(root, adj, n)
---------------------------------------------------------------------------
--- if len(c) == 1:
------- return (rootcode(c[0], adj, -1, 0, pr) << 1) | 1
--- else:
------- return (rootcode(c[0], adj, -1, 0, pr) *
--------------- rootcode(c[1], adj, -1, 0, pr)) << 1
---------------------------------------------------------------------------
def isomorphic(r1, adj1, r2, adj2, n1, n2, pr, rooted=False):
--- if rooted:
------- return rootcode(r1, adj1, -1, 0, pr) == rootcode(r2, adj2, -1,
↪  0, pr)
--- return treecode(r1, adj1, n1, pr) == treecode(r2, adj2, n2, pr) --
```

5. Math I Algebra

5.0.1. *Generating Function Manager.*

```
MOD = 998244353
---------------------------------------------------------------------------
def mod_pow(base, exp):
--- res = 1
--- base %= MOD
--- while exp:
------- if exp & 1:
----------- res = res * base % MOD
------- base = base * base % MOD
------- exp >>= 1
--- return res
---------------------------------------------------------------------------
---------------------------------------------------------------------------
class GF_Manager:
--- DEPTH = 23
---------------------------------------------------------------------------
--- def __init__(self):
------- self.prim = [0] * (self.DEPTH + 1)
------- self.prim_inv = [0] * (self.DEPTH + 1)
------- self.two_inv = [0] * (self.DEPTH + 1)
------- self.set_up_primitives()
---------------------------------------------------------------------------
--- def set_up_primitives(self):
------- self.prim[self.DEPTH] = 31
------- self.prim_inv[self.DEPTH] = mod_pow(self.prim[self.DEPTH], MOD
↪  - 2)
------- self.two_inv[self.DEPTH] = mod_pow(1 << self.DEPTH, MOD - 2) -
---------------------------------------------------------------------------
------- for n in range(self.DEPTH - 1, -1, -1):
----------- self.prim[n] = self.prim[n + 1] * self.prim[n + 1] % MOD -
----------- self.prim_inv[n] = mod_pow(self.prim[n], MOD - 2)
----------- self.two_inv[n] = mod_pow(1 << n, MOD - 2)
---------------------------------------------------------------------------
--- def NTT(self, A, n, is_inverse=False):
------- if n == 0:
----------- return A[:]
---------------------------------------------------------------------------
------- half = 1 << (n - 1)
------- even = A[0::2]
------- odd = A[1::2]
---------------------------------------------------------------------------
------- even = self.NTT(even, n - 1, is_inverse)
------- odd = self.NTT(odd, n - 1, is_inverse)
---------------------------------------------------------------------------
------- res = [0] * (1 << n)
------- w1 = self.prim_inv[n] if is_inverse else self.prim[n]
------- w = 1
---------------------------------------------------------------------------
------- for i in range(half):
----------- t = w * odd[i] % MOD
----------- res[i] = (even[i] + t) % MOD
----------- res[i + half] = (even[i] - t) % MOD
----------- w = w * w1 % MOD
---------------------------------------------------------------------------
------- return res
---------------------------------------------------------------------------
--- def mult(self, A, B):
------- an, bn = len(A), len(B)
------- degree = an + bn - 1
---------------------------------------------------------------------------
------- n = 0
------- while (1 << n) < degree:
----------- n += 1
---------------------------------------------------------------------------
------- size = 1 << n
------- tA = A + [0] * (size - an)
------- tB = B + [0] * (size - bn)
---------------------------------------------------------------------------
------- tA = self.NTT(tA, n)
------- tB = self.NTT(tB, n)
---------------------------------------------------------------------------
------- for i in range(size):
----------- tA[i] = tA[i] * tB[i] % MOD
---------------------------------------------------------------------------
------- tA = self.NTT(tA, n, True)
---------------------------------------------------------------------------
------- inv = self.two_inv[n]
------- return [(x * inv) % MOD for x in tA[:degree]]
---------------------------------------------------------------------------
--- def horners(self, F, x):
------- ans = 0
------- for coef in reversed(F):
----------- ans = (ans * x + coef) % MOD
------- return ans
```

5.0.2. *Fast Fourier Transform.*

```
import math
---------------------------------------------------------------------------
class Poly:
--- def __init__(self, a=0.0, b=0.0):
------- self.a = a
------- self.b = b
---------------------------------------------------------------------------
--- def __add__(self, other):
------- return Poly(self.a + other.a, self.b + other.b)
---------------------------------------------------------------------------
--- def __sub__(self, other):
------- return Poly(self.a - other.a, self.b - other.b)
---------------------------------------------------------------------------
--- def __mul__(self, other):
------- return Poly(
----------- self.a * other.a - self.b * other.b,
----------- self.a * other.b + self.b * other.a
------- )
---------------------------------------------------------------------------
--- def __repr__(self):
------- return f"({self.a}+{self.b}i)"
---------------------------------------------------------------------------
def fft_recursive(inp, n, s):
--- if n == 1:
------- return [inp[0]]
---------------------------------------------------------------------------
--- half = n // 2
---------------------------------------------------------------------------
--- even = fft_recursive(inp, half, s * 2)
--- odd = fft_recursive(inp[s:], half, s * 2)
---------------------------------------------------------------------------
--- res = [None] * n
--- w = Poly(1, 0)
--- wn = Poly(math.cos(math.pi / half), math.sin(math.pi / half))
---------------------------------------------------------------------------
--- for i in range(half):
------- t = w * odd[i]
------- res[i] = even[i] + t
------- res[i + half] = even[i] - t
------- w = w * wn
---------------------------------------------------------------------------
--- return res
---------------------------------------------------------------------------
def fft(p):
--- return fft_recursive(p, len(p), 1)
---------------------------------------------------------------------------
def inverse_fft(p):
--- n = len(p)
---------------------------------------------------------------------------
--- p = [Poly(x.a, -x.b) for x in p]
---------------------------------------------------------------------------
--- p = fft(p)
---------------------------------------------------------------------------
--- res = []
```

```
    for x in p:
        res.append(Poly(x.a / n, -x.b / n))

    return res
```

5.0.3. *Fft Polynomial Multiplication.*

```
def fft_multiply(a, b):
    an, bn = len(a), len(b)
    degree = an + bn - 1

    n = 1
    while n < degree:
        n <<= 1

    A = [Poly(x, 0) for x in a] + [Poly(0, 0)] * (n - an)
    B = [Poly(x, 0) for x in b] + [Poly(0, 0)] * (n - bn)

    A = fft(A)
    B = fft(B)

    for i in range(n):
        A[i] = A[i] * B[i]

    A = inverse_fft(A)

    result = [int(round(A[i].a)) for i in range(degree)]
    return result
```

5.0.4. *Polynomial Long Division.*

```
MOD = 998244353
G = 3

def mod_pow(a, e):
    r = 1
    while e:
        if e & 1:
            r = r * a % MOD
        a = a * a % MOD
        e >>= 1
    return r

def bit_reverse(a):
    n = len(a)
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]

def ntt(a, invert=False):
    n = len(a)
    bit_reverse(a)

    length = 2
    while length <= n:
        wlen = mod_pow(G, (MOD - 1) // length)
        if invert:
            wlen = mod_pow(wlen, MOD - 2)

        for i in range(0, n, length):
            w = 1
            half = length // 2
            for j in range(i, i + half):
                u = a[j]
                v = a[j + half] * w % MOD
                a[j] = (u + v) % MOD
                a[j + half] = (u - v) % MOD
                w = w * wlen % MOD

        length <<= 1

    if invert:
        inv_n = mod_pow(n, MOD - 2)
        for i in range(n):
            a[i] = a[i] * inv_n % MOD

def multiply(a, b):
    n = 1
    while n < len(a) + len(b):
        n <<= 1

    fa = a[:] + [0] * (n - len(a))
    fb = b[:] + [0] * (n - len(b))

    ntt(fa)
    ntt(fb)

    for i in range(n):
        fa[i] = fa[i] * fb[i] % MOD

    ntt(fa, True)

    return fa[:len(a) + len(b) - 1]

def poly_inv(f, n):
    if n == 1:
        return [mod_pow(f[0], MOD - 2)]

    m = (n + 1) // 2
    g = poly_inv(f, m)

    size = 1
    while size < 2 * n:
        size <<= 1

    a = f[:n] + [0] * (size - len(f[:n]))
    g = g + [0] * (size - len(g))

    ntt(a)
    ntt(g)

    for i in range(size):
        g[i] = g[i] * (2 - a[i] * g[i] % MOD) % MOD

    ntt(g, True)

    return g[:n]

def poly_divide(A, B):
    n, m = len(A), len(B)

    if n < m:
        return [], A[:]

    k = n - m + 1

    A_rev = A[::-1]
    B_rev = B[::-1]

    B_inv = poly_inv(B_rev, k)

    Q_rev = multiply(A_rev, B_inv)[:k]
    Q = Q_rev[::-1]

    BQ = multiply(B, Q)
    R = [(A[i] - (BQ[i] if i < len(BQ) else 0)) % MOD for i in range(n)]

    while Q and Q[-1] == 0:
        Q.pop()
    while R and R[-1] == 0:
        R.pop()

    return Q, R
```

5.0.5. *Matrix Multiplication.*

```
MOD = 998244353

def multiply(A, B):
    p = len(A)
    q = len(A[0])
    r = len(B[0])

    if len(B) != q:
        raise ValueError("Dimension mismatch")

    AB = [[0] * r for _ in range(p)]

    for i in range(p):
        for j in range(q):
            for k in range(r):
                AB[i][k] = (AB[i][k] + A[i][j] * B[j][k]) % MOD

    return AB
```

5.0.6. *Matrix Power.*

```
MOD = 998244353

def multiply(A, B):
    p, q, r = len(A), len(A[0]), len(B[0])

    res = [[0] * r for _ in range(p)]

    for i in range(p):
        for j in range(q):
            for k in range(r):
                res[i][k] = (res[i][k] + A[i][j] * B[j][k]) % MOD

    return res

def identity(n):
    I = [[0] * n for _ in range(n)]
    for i in range(n):
        I[i][i] = 1
    return I

def power(B, e):
    n = len(B)

    result = identity(n)
    base = [row[:] for row in B]

    while e > 0:
        if e & 1:
            result = multiply(result, base)
        base = multiply(base, base)
        e >>= 1

    return result
```

5.0.7. *Gauss Jordan Matrix Determinant.*

```
EPS = 1e-9

def gauss_jordan(A):
    n = len(A)
```

```
--- m = len(A[0])
---
--- singular = False
---
--- i = 0
--- for p in range(m):
------- if i >= n:
----------- break
---
------- pivot = i
------- for k in range(i + 1, n):
----------- if abs(A[k][p]) > abs(A[pivot][p]):
--------------- pivot = k
---
------- if abs(A[pivot][p]) < EPS:
----------- singular = True
----------- continue
---
------- A[i], A[pivot] = A[pivot], A[i]
---
------- div = A[i][p]
------- for j in range(p, m):
----------- A[i][j] /= div
---
------- for k in range(n):
----------- if k == i:
--------------- continue
----------- factor = A[k][p]
----------- for j in range(p, m):
--------------- A[k][j] -= factor * A[i][j]
---
------- i += 1
---
--- return not singular
---
def apply(A, x):
--- return [sum(A[i][j] * x[j] for j in range(len(x))) for i in
↪  range(len(A))]
```

6. Math II Combinatorics

6.0.1. *Lucas Theorem.*

```
_fact_cache = []
_last_p = -1
---
def lucas(n, k, p):
--- if k < 0 or k > n:
------- return 0
--- if k == 0 or n == k:
------- return 1
---
--- global _fact_cache, _last_p
--- if _last_p != p:
------- _fact_cache = [1] * p
------- for i in range(2, p):
----------- _fact_cache[i] = (_fact_cache[i - 1] * i) % p
------- _last_p = p
---
--- res_num = 1
--- res_den = 1
---
--- while n > 0 or k > 0:
------- ni, ki = n % p, k % p
------- if ki > ni:
----------- return 0
------- res_num = (res_num * _fact_cache[ni]) % p
------- res_den = (res_den * _fact_cache[ki] * _fact_cache[ni - ki]) %
↪  p
------- n //= p
------- k //= p
--- return (res_num * pow(res_den, p - 2, p)) % p
---
# Input Format: print(lucas(n, k, p))
```

6.0.2. *Catalan Numbers.*

```
def catalan(n):
--- if (n == 0 or n == 1):
------- return 1
---
--- dp = [0] * (n + 1)
--- dp[0] = dp[1] = 1
---
--- for i in range(2, n + 1):
------- for j in range(i):
----------- dp[i] += dp[j] * dp[i-j-1]
---
--- return dp[n]
---
# for i in range(10):
#     print(catalan(i))
```

6.0.3. *Kth Permutation.*

```
#fact = [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880]
---
def findKthPermutation(n, k):
--- nums = [str(i) for i in range(1, n + 1)]
---
--- fact = [1] * n
--- for i in range(1, n):
------- fact[i] = fact[i - 1] * i
---
--- k -= 1
--- res = []
---
--- for i in range(n - 1, -1, -1):
------- idx = k // fact[i]
------- k %= fact[i]
---
------- res.append(nums.pop(idx))
---
--- return ''.join(res)
```

6.0.4. *Factoradics.*

```
def factoradic(arr):
--- n = len(arr)
--- ft = FenwickTree(n)
--- for i in range(1, n + 1):
------- ft.add(i, 1)
---
--- res = []
--- for val in arr:
------- bit_val = val + 1
------- s = ft.query(bit_val - 1)
------- ft.add(bit_val, -1)
------- res.append(s)
--- return res
---
def permute(fact, n):
--- ft = FenwickTree(n)
--- for i in range(1, n + 1):
------- ft.add(i, 1)
---
--- res = []
--- for val in fact:
------- idx = ft.find_kth(val)
------- ft.add(idx, -1)
------- res.append(idx - 1)
--- return res
```

7. Math III Number Theory

7.0.1. *Gcd Subset Counter.*

```
def gcd_subset_count(arr):
--- n = len(arr)
--- mx = max(arr)
---
--- f = [0] * (mx + 1)
--- for x in arr:
------- f[x] += 1
---
--- gcnt = [0] * (mx + 1)
---
--- for i in range(mx, 0, -1):
------- add = 0
---
------- for j in range(i, mx + 1, i):
----------- add += f[j]
---
------- total = (1 << add) - 1  # 2^add - 1
---
------- sub = 0
------- for j in range(2 * i, mx + 1, i):
----------- sub += gcnt[j]
---
------- gcnt[i] = total - sub
---
--- return gcnt
```

7.0.2. *Linear Diophantine.*

```
def gcd(a, b):
--- if a == 0:
------- return b
--- return gcd(b % a, a)
---
def isPossible(a, b, c):
--- g = gcd(a, b)
---
--- return c % g == 0
```

7.0.3. *Chinese Remainder Theorem.*

```
def gcd_extended(a, b):
--- if a == 0:
------- return b, 0, 1
--- gcd, x1, y1 = gcd_extended(b % a, a)
--- x = y1 - (b // a) * x1
--- y = x1
--- return gcd, x, y
---
def find_min_x(num, rem):
--- prod = 1
--- for n in num:
------- prod *= n
---
--- result = 0
--- for i in range(len(num)):
------- prod_i = prod // num[i]
------- _, inv_i, _ = gcd_extended(prod_i, num[i])
------- result += rem[i] * prod_i * inv_i
---
--- return result % prod
```

7.0.4. *Euler Totient.*

```
def totient(n):
    if n <= 1:
        return 1

    tot = n
    i = 2

    while i * i <= n:
        if n % i == 0:
            tot -= tot // i
            while n % i == 0:
                n //= i
        i += 1

    if n > 1:
        tot -= tot // n

    return tot
```

7.0.5. *Modular Exponentiation.*

```
def mod_pow(b, e, m):
    res = 1 % m

    while e > 0:
        if e & 1:
            res = (res * b) % m
        b = (b * b) % m
        e >>= 1

    return res
```

7.0.6. *Mod Inverse.*

```
def extended_euclid(a, b):
    if b == 0:
        return a, 1, 0

    g, x1, y1 = extended_euclid(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return g, x, y


def modinv(a, m):
    g, x, y = extended_euclid(a, m)

    if g != 1:
        return 0  # no inverse

    return x % m
```

7.0.7. *Modulo Solver.*

```
def extended_euclid(a, b):
    if b == 0:
        return a, 1, 0

    g, x1, y1 = extended_euclid(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return g, x, y


def modsolver(a, b, m):
    g, x, y = extended_euclid(a, m)

    if b % g != 0:
        return (-1, -1)

    m_reduced = m // g

    x0 = (x * (b // g)) % m_reduced

    return (x0, m_reduced)
```

7.0.8. *Primitive Root.*

```
def primitive_root(m):
    if m == 2:
        return 1

    phi = m - 1

    factors = set()
    x = phi
    i = 2

    while i * i <= x:
        if x % i == 0:
            factors.add(i)
            while x % i == 0:
                x //= i
        i += 1

    if x > 1:
        factors.add(x)

    for g in range(2, m):
        ok = True
        for f in factors:
            if mod_pow(g, phi // f, m) == 1:
                ok = False
                break
        if ok:
            return g

    return -1
```

8. Math IV Numerical Methods

8.0.1. *Fast Square Testing.*

```
import math

class FastSquareTester:
    def __init__(self):
        self.M = 0
        self.init_is_square()

    def init_is_square(self):
        for i in range(64):
            self.M |= (1 << (i * i % 64))

    def is_square(self, x):
        if x < 0: return False
        if x == 0: return True

        if not (self.M & (1 << (x & 63))):
            return False

        c = (x & -x).bit_length() - 1
        if c & 1:
            return False

        temp_x = x >> c
        if (temp_x & 7) != 1:
            return False

        r = int(math.isqrt(temp_x))
        return r * r == temp_x

tester = FastSquareTester()
```

8.0.2. *Simpson Integration.*

```
def f(x):
    return x ** 2
def simpson_integration(a, b, n=1000000):
    if n % 2 != 0:
        n += 1

    h = (b - a) / n
    s = f(a) + f(b)

    for i in range(1, n):
        x = a + h * i
        if i % 2 == 1:
            s += 4 * f(x)
        else:
            s += 2 * f(x)

    return s * (h / 3)
```

9. Strings

9.1. Longest Common Prefix.

```
def build_lcp(s, sa):
    n = len(s)
    rank = [0] * n
    lcp = [0] * n

    for i in range(n):
        rank[sa[i]] = i

    k = 0
    for i in range(n):
        if rank[i] == 0:
            k = 0
            continue

        j = sa[rank[i] - 1]

        while i + k < n and j + k < n and s[i + k] == s[j + k]:
            k += 1

        lcp[rank[i]] = k

        if k > 0:
            k -= 1

    return lcp
```

9.2. Hasher.

```
class Hasher:
    def __init__(self, s, bases=[31, 37], mod=10 ** 9 + 7):
        self.bases = bases
        self.mod = mod
        self.n = len(s)
        self.num_hashes = len(bases)
        self.p_pow = [[1] * (self.n + 1) for _ in
↪   range(self.num_hashes)]
        self.h_ans = [[0] * (self.n + 1) for _ in
↪   range(self.num_hashes)]

        for i, b in enumerate(self.bases):
            for j in range(self.n):
                self.p_pow[i][j + 1] = (self.p_pow[i][j] * b) %
↪   self.mod
```

```
---------------- self.h_ans[i][j + 1] = (self.h_ans[i][j] + s[j] *
↪  self.p_pow[i][j]) % self.mod
--------------------------------------------------------------------
--- def get_hash(self, l, r): ------------------------------------
------- return tuple((self.h_ans[i][r + 1] - self.h_ans[i][l]) %
↪  self.mod
--------------------- for i in range(self.num_hashes)) ----------------
--------------------------------------------------------------------
--- def are_equal(self, l1, r1, l2, r2): -------------------------
------- if (r1 - l1) != (r2 - l2): return False ----------------------
--------------------------------------------------------------------
------- h1 = self.get_hash(l1, r1) -----------------------------------
------- h2 = self.get_hash(l2, r2) -----------------------------------
--------------------------------------------------------------------
------- return all((h1[i] * self.p_pow[i][l2]) % self.mod == ---------
------------------- (h2[i] * self.p_pow[i][l1]) % self.mod ------------
------------------- for i in range(self.num_hashes)) ------------------
```

9.3. Z Algorithm.

9.4. Aho Corasick Trie.

```
from collections import deque ----------------------------------------
import random --------------------------------------------------------
import string --------------------------------------------------------
--------------------------------------------------------------------
class Node: ----------------------------------------------------------
--- def __init__(self): ------------------------------------------
------- self.next = {} -----------------------------------------
------- self.fail = None ---------------------------------------
------- self.count = 0 -----------------------------------------
--------------------------------------------------------------------
--- def add(self, s): --------------------------------------------
------- node = self --------------------------------------------
------- for c in s: --------------------------------------------
----------- if c not in node.next: -------------------------------
--------------- node.next[c] = Node() ------------------------------
----------- node = node.next[c] ----------------------------------
------- node.count += 1 ----------------------------------------
--------------------------------------------------------------------
--- def prepare(self): -------------------------------------------
------- root = self --------------------------------------------
------- q = deque() --------------------------------------------
--------------------------------------------------------------------
------- root.fail = root ---------------------------------------
--------------------------------------------------------------------
------- for child in root.next.values(): -----------------------
----------- child.fail = root ------------------------------------
----------- q.append(child) --------------------------------------
--------------------------------------------------------------------
------- while q: -----------------------------------------------
----------- head = q.popleft() -----------------------------------
--------------------------------------------------------------------
----------- for c, nxt in head.next.items(): ---------------------
--------------- f = head.fail ----------------------------------
--------------------------------------------------------------------
--------------- while f != root and c not in f.next: -----------
------------------- f = f.fail -------------------------------
--------------------------------------------------------------------
--------------- nxt.fail = f.next[c] if c in f.next else root --------
--------------------------------------------------------------------
--------------- nxt.count += nxt.fail.count --------------------
--------------------------------------------------------------------
--------------- q.append(nxt) ----------------------------------
--------------------------------------------------------------------
--- def search(self, s): -----------------------------------------
------- root = self --------------------------------------------
------- p = self -----------------------------------------------
------- ans = 0 ------------------------------------------------
--------------------------------------------------------------------
------- for c in s: --------------------------------------------
----------- while p != root and c not in p.next: ----------------
--------------- p = p.fail -------------------------------------
--------------------------------------------------------------------
----------- if c in p.next: --------------------------------------
--------------- p = p.next[c] ----------------------------------
--------------- ans += p.count ---------------------------------
--------------------------------------------------------------------
------- return ans ---------------------------------------------
```

9.5. Palindromes.

9.5.1. *Palindromic Tree.*

```
import sys -----------------------------------------------------------
--------------------------------------------------------------------
N = 200000 -----------------------------------------------------------
MAX = N * 2 + 5 ------------------------------------------------------
--------------------------------------------------------------------
par = [-1] * MAX -----------------------------------------------------
len_arr = [0] * MAX --------------------------------------------------
node = [0] * MAX -----------------------------------------------------
cs = [0] * MAX -------------------------------------------------------
cnt = [0] * (N + 5) --------------------------------------------------
--------------------------------------------------------------------
child = [[-1] * 128 for _ in range(MAX)] -----------------------------
size = 0 -------------------------------------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def newNode(p=-1): ---------------------------------------------------
--- global size --------------------------------------------------
--- cnt[size] = 0 ------------------------------------------------
--- par[size] = p ------------------------------------------------
--- len_arr[size] = 0 if p == -1 else len_arr[p] + 2 -------------
--------------------------------------------------------------------
--- for i in range(128): -----------------------------------------
------- child[size][i] = -1 ------------------------------------
--------------------------------------------------------------------
--- size += 1 ----------------------------------------------------
--- return size - 1 ----------------------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def get(i, c): -------------------------------------------------------
--- if child[i][c] == -1: ----------------------------------------
------- child[i][c] = newNode(i) -------------------------------
--- return child[i][c] -------------------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def manachers(s): ----------------------------------------------------
--- global size --------------------------------------------------
--------------------------------------------------------------------
--- n = len(s) ---------------------------------------------------
--- cn = n * 2 + 1 -----------------------------------------------
--------------------------------------------------------------------
--- for i in range(n): -------------------------------------------
------- cs[i * 2] = -1 -----------------------------------------
------- cs[i * 2 + 1] = ord(s[i]) ------------------------------
-------  ----------------------------------------------------------
--- size = 0 -----------------------------------------------------
--------------------------------------------------------------------
--- odd = newNode() ----------------------------------------------
--- even = newNode() ---------------------------------------------
--------------------------------------------------------------------
--- len_arr[odd] = -1 --------------------------------------------
--------------------------------------------------------------------
--- cen = 0 ------------------------------------------------------
--- rad = 0 ------------------------------------------------------
--------------------------------------------------------------------
--- for i in range(cn): ------------------------------------------
------- node[i] = even if i % 2 == 0 else get(odd, cs[i]) -------
--------------------------------------------------------------------
--- for i in range(1, cn): ---------------------------------------
------- if i > rad: --------------------------------------------
----------- L = i - 1 --------------------------------------------
----------- R = i + 1 --------------------------------------------
------- else: --------------------------------------------------
----------- M = cen * 2 - i --------------------------------------
----------- node[i] = node[M] ------------------------------------
--------------------------------------------------------------------
----------- if len_arr[node[M]] < rad - i: -----------------------
--------------- L = -1 -----------------------------------------
--------------- R = 0 ------------------------------------------
----------- else: ------------------------------------------------
--------------- R = rad + 1 ------------------------------------
--------------- L = i * 2 - R ----------------------------------
--------------------------------------------------------------------
--------------- while len_arr[node[i]] > rad - i: --------------
------------------- node[i] = par[node[i]] -------------------
--------------------------------------------------------------------
------- while L >= 0 and R < cn and cs[L] == cs[R]: ------------
----------- if cs[L] != -1: --------------------------------------
--------------- node[i] = get(node[i], cs[L]) ------------------
----------- L -= 1 -----------------------------------------------
----------- R += 1 -----------------------------------------------
--------------------------------------------------------------------
------- cnt[node[i]] += 1 --------------------------------------
--------------------------------------------------------------------
------- if i + len_arr[node[i]] > rad: -------------------------
----------- rad = i + len_arr[node[i]] ---------------------------
----------- cen = i ----------------------------------------------
--------------------------------------------------------------------
--- for i in range(size - 1, -1, -1): ----------------------------
------- if par[i] != -1: ---------------------------------------
----------- cnt[par[i]] += cnt[i] --------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def countUniquePalindromes(s): ---------------------------------------
--- manachers(s) -------------------------------------------------
--- return size --------------------------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def countAllPalindromes(s): ------------------------------------------
--- manachers(s) -------------------------------------------------
--- return sum(cnt[:size]) ---------------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
def longestPalindrome(s): --------------------------------------------
--- manachers(s) -------------------------------------------------
--------------------------------------------------------------------
--- n = len(s) ---------------------------------------------------
--- cn = n * 2 + 1 -----------------------------------------------
--------------------------------------------------------------------
--- mx = 0 -------------------------------------------------------
--- for i in range(1, cn): ---------------------------------------
------- if len_arr[node[mx]] < len_arr[node[i]]: ---------------
----------- mx = i -----------------------------------------------
--------------------------------------------------------------------
--- pos = (mx - len_arr[node[mx]]) // 2 --------------------------
--- return s[pos:pos + len_arr[node[mx]]] ------------------------
```

9.5.2. *Eertree.*

```
class Node: ----------------------------------------------------------
--- def __init__(self, start=0, end=0, length=0, link=0): ----------
------- self.start = start -------------------------------------
------- self.end = end -----------------------------------------
------- self.len = length --------------------------------------
------- self.link = link ---------------------------------------
------- self.next = [0] * 26 -----------------------------------
--------------------------------------------------------------------
--------------------------------------------------------------------
class Eertree: -------------------------------------------------------
--- def __init__(self): ------------------------------------------
------- self.tree = [] -----------------------------------------
--------------------------------------------------------------------
```

```
        self.tree.append(Node())
        self.tree.append(Node(0, 0, -1, 1))
        self.tree.append(Node(0, 0, 0, 1))

        self.cur = 1
        self.ptr = 2

    def get_link(self, v, s, i):
        while True:
            cur_len = self.tree[v].len
            if i - cur_len - 1 >= 0 and s[i] == s[i - cur_len - 1]:
                return v
            v = self.tree[v].link

    def add_char(self, s, i):
        cur = self.cur
        cur = self.get_link(cur, s, i)

        c = ord(s[i]) - ord('a')

        if self.tree[cur].next[c]:
            self.cur = self.tree[cur].next[c]
            return

        self.ptr += 1
        self.tree[cur].next[c] = self.ptr

        length = self.tree[cur].len + 2
        self.tree.append(Node(i - length + 1, i, length, 0))

        new_node = self.ptr

        if self.tree[new_node].len == 1:
            self.tree[new_node].link = 2
        else:
            tmp = self.tree[cur].link
            tmp = self.get_link(tmp, s, i)
            self.tree[new_node].link = self.tree[tmp].next[c]

        self.cur = new_node

    def build(self, s):
        for i in range(len(s)):
            self.add_char(s, i)
```

9.6. Trie.

```
class Node:
    def __init__(self):
        self.children = {}
        self.prefixes = 0
        self.words = 0

class Trie:
    def __init__(self):
        self.root = Node()

    def insert(self, seq):
        cur = self.root

        for i in range(len(seq) + 1):
            cur.prefixes += 1

            if i == len(seq):
                cur.words += 1
                break

            head = seq[i]

            if head not in cur.children:
                cur.children[head] = Node()

            cur = cur.children[head]

    def countMatches(self, seq):
        cur = self.root

        for c in seq:
            if c not in cur.children:
                return 0
            cur = cur.children[c]

        return cur.words

    def countPrefixes(self, seq):
        cur = self.root

        for c in seq:
            if c not in cur.children:
                return 0
            cur = cur.children[c]

        return cur.prefixes
```

9.7. Suffix Array.

```
class SuffixArray:
    def __init__(self):
        self.n = 0
        self.suffix = []
        self.equiv = []
        self.equiv_pair = []
        self.s = ""

    def build(self, s):
        if not s.endswith('$'):
            s += '$'

        self.s = s
        self.n = len(s)

        self.suffix = list(range(self.n))
        self.equiv = [0] * self.n
        self.equiv_pair = [(0, 0)] * self.n

        self.suffix.sort(key=lambda i: s[i])

        cls = 0
        for i in range(self.n):
            if i == 0 or s[self.suffix[i]] != s[self.suffix[i - 1]]:
                cls += 1
            self.equiv[self.suffix[i]] = cls

        k = 1
        while k < self.n:
            for i in range(self.n):
                self.equiv_pair[i] = (
                    self.equiv[i],
                    self.equiv[(i + k) % self.n]
                )

            self.suffix.sort(key=lambda i: self.equiv_pair[i])

            cls = 0
            new_equiv = [0] * self.n

            for i in range(self.n):
                if i == 0 or self.equiv_pair[self.suffix[i]] !=
↪   self.equiv_pair[self.suffix[i - 1]]:
                    cls += 1
                new_equiv[self.suffix[i]] = cls

            self.equiv = new_equiv
            k <<= 1

    def _cmp(self, start, pattern):
        s = self.s
        n = len(s)

        i = start
        j = 0

        while i < n and j < len(pattern):
            if s[i] != pattern[j]:
                return ord(s[i]) - ord(pattern[j])
            i += 1
            j += 1

        if j == len(pattern):
            return 0
        return -1

    def lower_bound(self, pattern):
        l, r = 0, self.n - 1
        res = self.n

        while l <= r:
            m = (l + r) // 2
            if self._cmp(self.suffix[m], pattern) >= 0:
                res = m
                r = m - 1
            else:
                l = m + 1

        return res

    def upper_bound(self, pattern):
        l, r = 0, self.n - 1
        res = self.n

        while l <= r:
            m = (l + r) // 2
            if self._cmp(self.suffix[m], pattern) > 0:
                res = m
                r = m - 1
            else:
                l = m + 1

        return res

    def count_occurrences(self, pattern):
        L = self.lower_bound(pattern)
        R = self.upper_bound(pattern)
        return max(0, R - L)
```

10. Other Algorithms

10.1. Longest Increasing Subsequence.

```
def lis(arr):
    if not arr:
        return []

    n = len(arr)
    seq = []
    back = [0] * n

    for i in range(n):
        lo, hi = 0, len(seq)
        while lo < hi:
            mid = (lo + hi) // 2
            if arr[seq[mid]] < arr[i]:
                lo = mid + 1
            else:
```

```
---------------- hi = mid ----------------------------------------------
------------------------------------------------------------------------
-------- res = lo ----------------------------------------------------
-------- if res < len(seq): ------------------------------------------
------------ seq[res] = i ------------------------------------------
-------- else: -------------------------------------------------------
------------ seq.append(i) -----------------------------------------
-------- back[i] = -1 if res == 0 else seq[res - 1] ------------------
------------------------------------------------------------------------
--- ans = [] ---------------------------------------------------------
--- at = seq[-1] -----------------------------------------------------
--- while at != -1: --------------------------------------------------
-------- ans.append(at) ----------------------------------------------
-------- at = back[at] -----------------------------------------------
------------------------------------------------------------------------
--- return ans[::-1] -------------------------------------------------
```

10.2. Cycle Find.

```
def find_cycle(x0, f):
--- t = f(x0)
--- h = f(f(x0))

--- while t != h:
-------- t = f(t)
-------- h = f(f(h))

--- mu = 0
--- h = x0
--- while t != h:
-------- t = f(t)
-------- h = f(h)
-------- mu += 1

--- lam = 1
--- h = f(t)
--- while t != h:
-------- h = f(h)
-------- lam += 1

--- return mu, lam
```

10.3. Stable Marriage.

```
from collections import deque

def stable_marriage(n, m_pref, w_pref):
--- w_rank = [[0] * n for _ in range(n)]

--- for w in range(n):
-------- for i, man in enumerate(w_pref[w]):
------------ w_rank[w][man] = i

--- q = deque(range(n))

--- at = [0] * n

--- eng = [-1] * n

--- res = [-1] * n

--- while q:
-------- m = q.popleft()

-------- while at[m] < n:
------------ w = m_pref[m][at[m]]
------------ at[m] += 1

------------ if eng[w] == -1:
---------------- eng[w] = m
---------------- res[m] = w
---------------- break

------------ if w_rank[w][m] < w_rank[w][eng[w]]:
---------------- q.append(eng[w])
---------------- eng[w] = m
---------------- res[m] = w
---------------- break

--- return res
```

10.4. Dates.

```
def c_div(a, b):
--- return int(a / b)

def date_to_int(y, m, d):
--- k = c_div(m - 14, 12)

--- return (
-------- 1461 * (y + 4800 + k) // 4
-------- + 367 * (m - 2 - k * 12) // 12
-------- - 3 * ((y + 4900 + k) // 100) // 4
-------- + d - 32075
--- )

def int_to_day(jd):
--- return jd % 7

def int_to_date(jd):
--- x = jd + 68569
--- n = 4 * x // 146097
--- x -= (146097 * n + 3) // 4

--- i = (4000 * (x + 1)) // 1461001
--- x -= 1461 * i // 4 - 31

--- j = 80 * x // 2447
--- d = x - 2447 * j // 80

--- x = j // 11
--- m = j + 2 - 12 * x
--- y = 100 * (n - 49) + i + x

--- return y, m, d
```

10.5. Simulated Annealing.

```
import random
import math
import time

def simulated_annealing(n, seconds):
--- rng = random.Random()

--- sol = list(range(1, n + 1))
--- rng.shuffle(sol)

--- def score(arr):
-------- return sum(abs(arr[i] - arr[i - 1]) for i in range(1, n))

--- cur = score(sol)

--- T0 = 100.0
--- T1 = 0.001
--- start = time.time()
--- iters = 0

--- while True:
-------- if iters % 16 == 0:
------------ progress = (time.time() - start) / seconds
------------ if progress >= 1.0:
---------------- break
------------ temp = T0 * (T1 / T0) ** progress

-------- a = rng.randint(0, n - 2)

-------- x, y = sol[a], sol[a + 1]

-------- delta = 0

-------- if a > 0:
------------ delta += abs(sol[a - 1] - y) - abs(sol[a - 1] - x)

-------- if a + 2 < n:
------------ delta += abs(x - sol[a + 2]) - abs(y - sol[a + 2])

-------- accept = delta <= 0 or rng.random() < math.exp(-delta / temp)

-------- if accept:
------------ sol[a], sol[a + 1] = sol[a + 1], sol[a]
------------ cur += delta

-------- iters += 1

--- return cur, sol
```

10.6. Simplex.

```
import math

EPS = 1e-9

class LPSolver:
--- def __init__(self, A, b, c):
-------- self.m = len(b)
-------- self.n = len(c)
-------- self.N = [0] * (self.n + 1)
-------- self.B = [0] * self.m
-------- self.D = [[0.0] * (self.n + 2) for _ in range(self.m + 2)]

-------- for i in range(self.m):
------------ for j in range(self.n):
---------------- self.D[i][j] = A[i][j]

-------- for i in range(self.m):
------------ self.B[i] = self.n + i
------------ self.D[i][self.n] = -1
------------ self.D[i][self.n + 1] = b[i]

-------- for j in range(self.n):
------------ self.N[j] = j
------------ self.D[self.m][j] = -c[j]

-------- self.N[self.n] = -1
-------- self.D[self.m + 1][self.n] = 1

--- def pivot(self, r, s):
-------- inv = 1.0 / self.D[r][s]

-------- for i in range(self.m + 2):
------------ if i != r:
---------------- for j in range(self.n + 2):
-------------------- if j != s:
------------------------ self.D[i][j] -= self.D[r][j] * self.D[i][s] *
↪  inv

-------- for j in range(self.n + 2):
------------ if j != s:
---------------- self.D[r][j] *= inv

-------- for i in range(self.m + 2):
------------ if i != r:
---------------- self.D[i][s] *= -inv
```

```
-------------------------------------------------------------------
------- self.D[r][s] = inv ----------------------------------------
------- self.B[r], self.N[s] = self.N[s], self.B[r] ----------------
-------------------------------------------------------------------
--- def simplex(self, phase): ---------------------------------------
------- x = self.m + 1 if phase == 1 else self.m ------------------
-------------------------------------------------------------------
------- while True: -----------------------------------------------
----------- s = -1 ------------------------------------------------
----------- for j in range(self.n + 1): ---------------------------
--------------- if phase == 2 and self.N[j] == -1: ----------------
------------------- continue ------------------------------------
--------------- if (s == -1 or ------------------------------------
------------------- self.D[x][j] < self.D[x][s] or ---------------
------------------- (abs(self.D[x][j] - self.D[x][s]) < EPS and
↪  self.N[j] < self.N[s])):
------------------- s = j ---------------------------------------
-------------------------------------------------------------------
----------- if self.D[x][s] > -EPS: -------------------------------
--------------- return True ---------------------------------------
-------------------------------------------------------------------
----------- r = -1 ------------------------------------------------
----------- for i in range(self.m): -------------------------------
--------------- if self.D[i][s] < EPS: ----------------------------
------------------- continue ------------------------------------
--------------- if (r == -1 or ------------------------------------
------------------- self.D[i][self.n + 1] / self.D[i][s] < -----------
------------------- self.D[r][self.n + 1] / self.D[r][s] or ----------
------------------- (abs(self.D[i][self.n + 1] / self.D[i][s] - ------
----------------------- self.D[r][self.n + 1] / self.D[r][s]) < EPS
↪  and
------------------- self.B[i] < self.B[r])): --------------------
------------------- r = i ---------------------------------------
-------------------------------------------------------------------
----------- if r == -1: -------------------------------------------
--------------- return False --------------------------------------
-------------------------------------------------------------------
----------- self.pivot(r, s) --------------------------------------
-------------------------------------------------------------------
--- def solve(self): ------------------------------------------------
------- x = [0.0] * self.n ----------------------------------------
-------------------------------------------------------------------
------- r = 0 -----------------------------------------------------
------- for i in range(1, self.m): --------------------------------
----------- if self.D[i][self.n + 1] < self.D[r][self.n + 1]: -------
--------------- r = i ---------------------------------------------
-------------------------------------------------------------------
------- if self.D[r][self.n + 1] < -EPS: --------------------------
----------- self.pivot(r, self.n) ---------------------------------
----------- if not self.simplex(1) or self.D[self.m + 1][self.n + 1] <
↪  -EPS:
--------------- return -math.inf, x -------------------------------
-------------------------------------------------------------------
----------- for i in range(self.m): -------------------------------
--------------- if self.B[i] == -1: -------------------------------
------------------- s = -1 --------------------------------------
------------------- for j in range(self.n + 1): -----------------
----------------------- if (s == -1 or ----------------------------
--------------------------- self.D[i][j] < self.D[i][s] or -------
--------------------------- (abs(self.D[i][j] - self.D[i][s]) < EPS and
↪  self.N[j] < self.N[s])):
--------------------------- s = j -------------------------------
------------------- self.pivot(i, s) ----------------------------
-------------------------------------------------------------------
------- if not self.simplex(2): -----------------------------------
----------- return math.inf, x ------------------------------------
-------------------------------------------------------------------
------- for i in range(self.m): -----------------------------------
----------- if self.B[i] < self.n: --------------------------------
--------------- x[self.B[i]] = self.D[i][self.n + 1] --------------
-------------------------------------------------------------------
------- return self.D[self.m][self.n + 1], x ----------------------
```

10.7. Fast Input Reading.

```
import sys ------------------------------------------------------
-------------------------------------------------------------------
data = sys.stdin.buffer.read().split() --------------------------
it = iter(data) -------------------------------------------------
-------------------------------------------------------------------
def next_int(): -------------------------------------------------
--- return int(next(it)) ------------------------------------------
```

10.8. Bit Hacks.

```
def snoob(x): ---------------------------------------------------
--- y = x & -x ------------------------------------------------
--- z = x + y -------------------------------------------------
--- return z | (((x ^ z) // y) >> 2) -------------------------
```

- **Legendre symbol:** $\left(\frac{a}{b}\right) = a^{(b-1)/2} \pmod b$
- **Heron's formula:** $\sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$
- **Pick's theorem:** $i + \frac{b}{2} - 1$
- **Euler's totient:** $n \prod_{p|n} \left(1 - \frac{1}{p}\right)$
- **König's theorem:**

$$|M_{\max}| = |VC_{\min}|$$

$$K = (L \setminus Z) \cup (R \cap Z)$$

- Steiner tree: $\leq n-2$ Steiner vertices
- Vertex cover / independent set:

$$|V| = \tau(G) + \alpha(G)$$

- **Lagrange polynomial:**

$$L(x) = \sum_{j=0}^{k} y_j \prod_{\substack{0 \leq m \leq k \\ m \neq j}} \frac{x - x_m}{x_j - x_m}$$

- **Hook length formula:**

$$d_\lambda = \frac{n!}{\prod h_\lambda(i,j)}$$

- **Möbius inversion:**

$$f(n) = \sum_{d|n} g(d), \quad g(n) = \sum_{d|n} \mu(d) f(n/d)$$

$$f(n) = \sum_{m=1}^{n} g(\lfloor n/m \rfloor), \quad g(n) = \sum_{m=1}^{n} \mu(m) f(\lfloor n/m \rfloor)$$

- Primitive Pythagorean triples:

$$\sim \frac{n}{2\pi}$$

- **Frobenius number:**

$$g(a_1, a_2) = a_1 a_2 - a_1 - a_2$$

$$N(a_1, a_2) = \frac{(a_1 - 1)(a_2 - 1)}{2}$$

$$g(da_1, da_2, a_3) = d\, g(a_1, a_2, a_3) + a_3(d-1)$$

10.9. Physics. Snell's law:

$$\frac{\sin\theta_1}{v_1} = \frac{\sin\theta_2}{v_2}$$

10.10. Markov Chains.

$$P^{(m+n)} = P^{(m)} P^{(n)}, \quad P^{(m)} = P^m$$

$$\mathbf{p}^{(0)} P^{(m)}$$

$$\gcd(R_i) = 1$$

$$\pi P = \pi$$

$$\pi_i = \frac{1}{\mathbb{E}[T_i]}$$

$$\lim_{m \to \infty} P^m = \pi$$

$$p_{uv} = \frac{w_{uv}}{\sum_x w_{ux}}$$

$$\pi_u = \frac{\sum_x w_{ux}}{\sum_v \sum_x w_{vx}}$$

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

$$N = (I - Q)^{-1}$$

$$N\mathbf{1}$$

$$NR$$

10.11. Primitive Roots.

$$\phi(\phi(n))$$

$$g^k$$

$$g^{i \cdot \phi(n)/k}$$

10.12. Floor.

$$\left\lfloor \frac{\lfloor x/y \rfloor}{z} \right\rfloor = \left\lfloor \frac{x}{yz} \right\rfloor$$

$$x \bmod y = x - y \left\lfloor \frac{x}{y} \right\rfloor$$

10.13. Maximum-Weight Closure.

$$(S, v, w), \quad (v, T, -w)$$

$$\infty$$

10.14. Tutte Matrix.

$$A_{ij} = x_{ij}, \quad A_{ji} = -x_{ij}$$

$$\det(A) = 0$$

10.15. Spanning Trees.

$$D - A$$

$$|\text{cofactor}(D - A)|$$

10.16. Möbius inversion.

$$g(n) = \sum_{d|n} f(d)$$

$$f(n) = \sum_{d|n} \mu(d)\, g(n/d)$$

10.17. Integer points.

$$\sum_{x=0}^{n} \left\lfloor \frac{C - Ax}{B} + 1 \right\rfloor$$

$$n = \left\lfloor \frac{c}{a} \right\rfloor$$

$$C - nA \geq 0$$